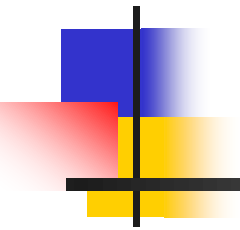
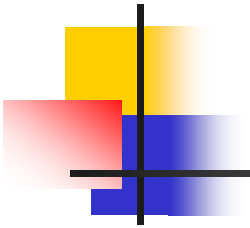


# Chapter 6

TREES



# 6.1 Introduction

- A tree consists of a finite set of elements, called **nodes**, and a finite set of directed lines, called **branches**, that connect the node.
- Trees can be classify as:
  - ✓ Static Trees – the form of trees has been determined.
  - ✓ Dynamic Trees – the form of trees is varying during the execution.

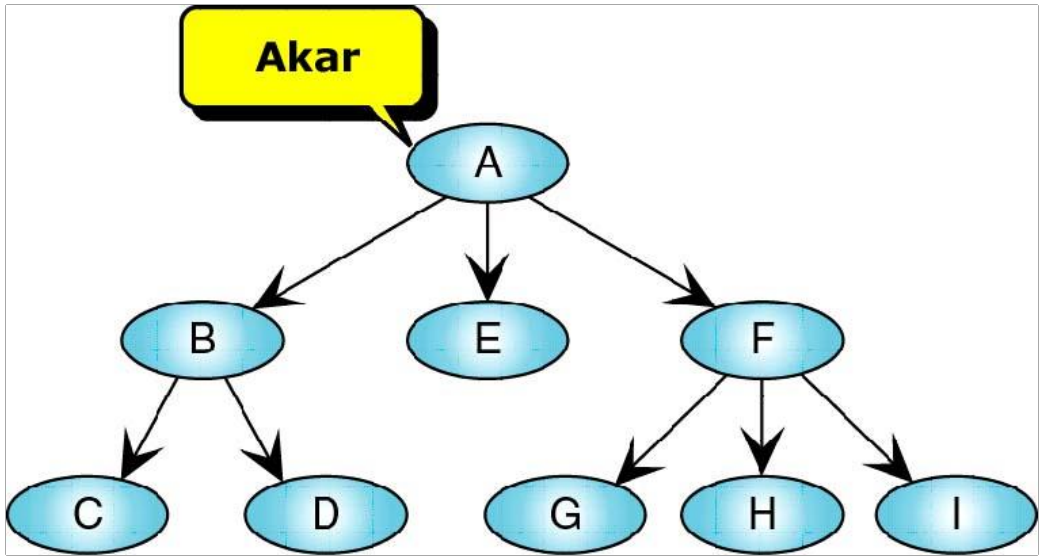
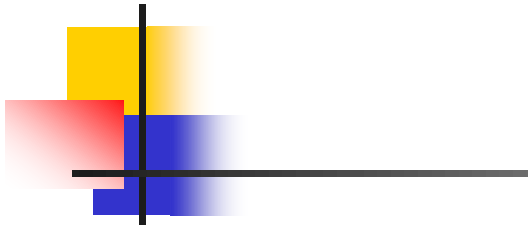
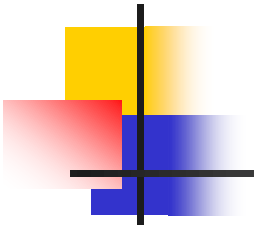


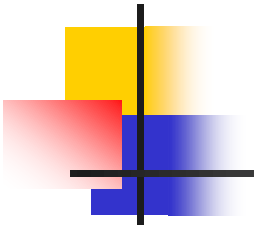

Fig. 1: A tree



- Basic Trees Anatomy – information from a tree are:
  - ✓ Family Relationship – parent node & child node.
  - ✓ Geometric Relationship – left /right/bottom/up.
  - ✓ Biological Name for tree – root, leaves, internal node, level.



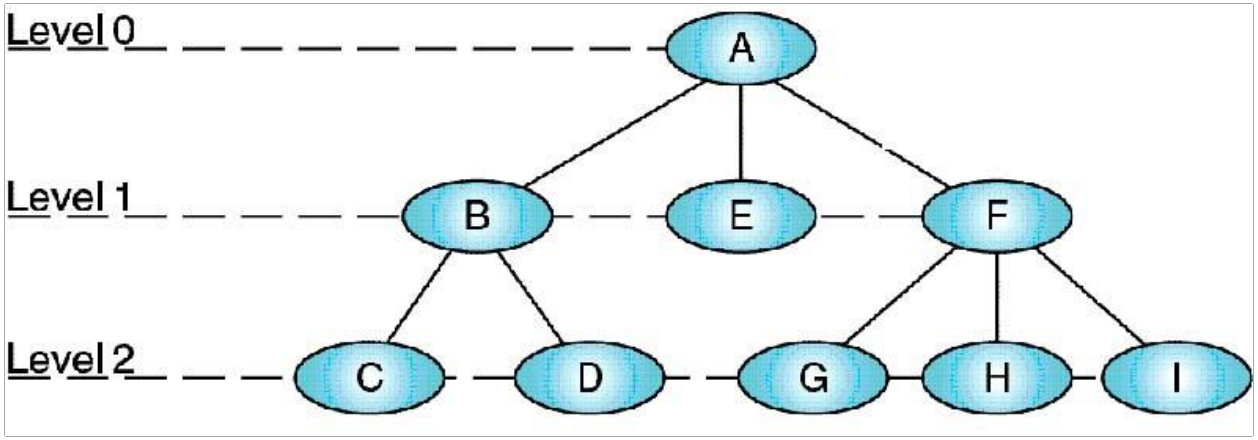
- The first node is called the **root**.
- A node is a **parent** if it is has successor nodes.
- A node with predecessor is a **child**.
  - ✓ A child node can be a left child node (left sub tree) or right child node (right sub tree).
- Two or more node with the same parent are **siblings**.
- A leaf node is a node without any child or empty sub trees.



- Nodes that are not a root or leaf are known as **internal nodes** because they are found in the middle portion of a tree.
- An ancestor is any node in the path from the node to the root.
- A descendant is any node in the path below the parent node; that is, all nodes in the paths from a given node to a leaf are descendants of the node.

# Cont. Introduction (6)

- Fig. 2 shows the usage of these terms.




| | | | |
|---|---|---|---|
| Root | : A | Leaves | : C, D, E, G, H, I |
| Parents | : A, B, F | Internal Nodes | : B, F |
| Children | : B, E, F, C, D, G, H, I | Descendants | : B – C & D |
| Siblings | : {B, E, F}, {C, D}, {G, H, I} | Ancestors | : C – B & A |

# 6.2 Binary Trees

- Binary tree is a tree in which no node can have more than two subtrees (a node can have zero, one, or two subtrees).

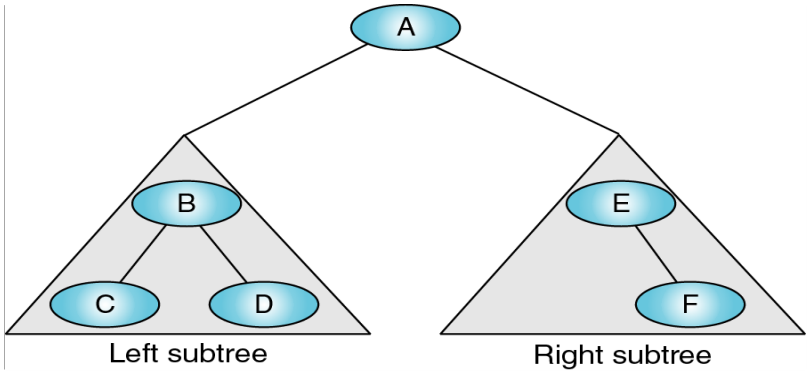



Fig. 3: Binary tree

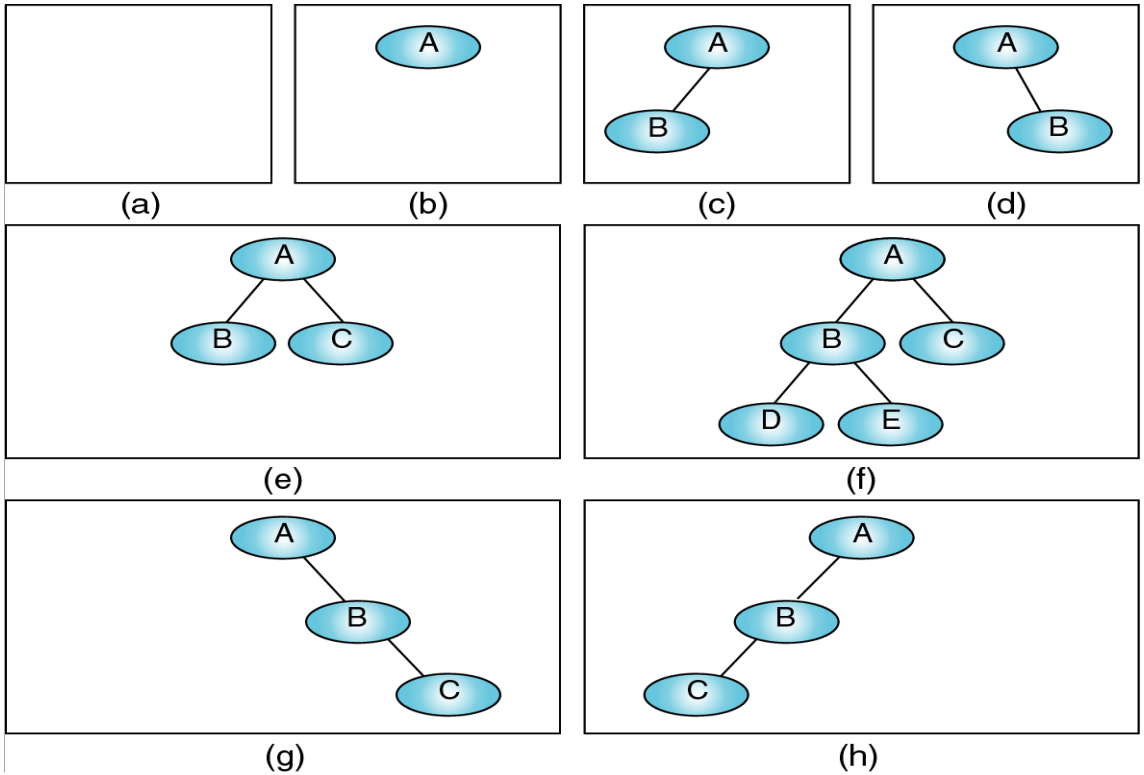
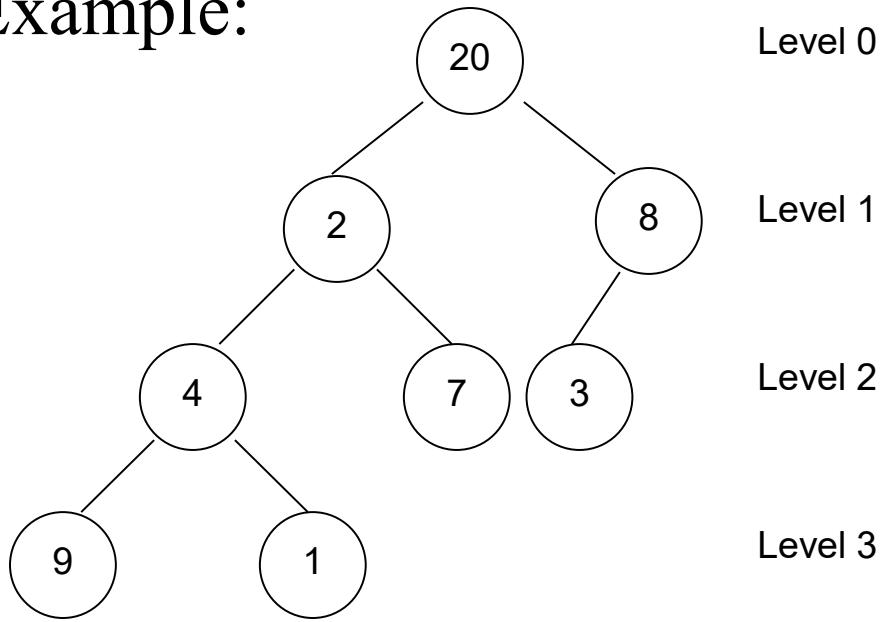
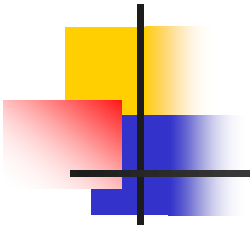


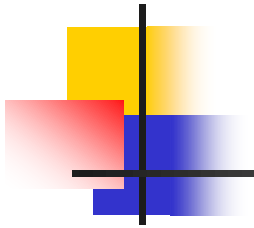

Fig. 4: A collection of binary trees

- A null tree is a tree with no node (see Fig. 4(a)).
- As you study this figure, note that symmetry (balance) is not a tree requirement.
- Example:

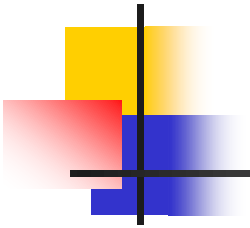




- Children 20 : 2, 8
  2 : 4, 7
  4 : 9, 1
  8 : 3
- Parents 4 : 2
  2 : 20
- Descendant 2 : 4, 9, 1, 7

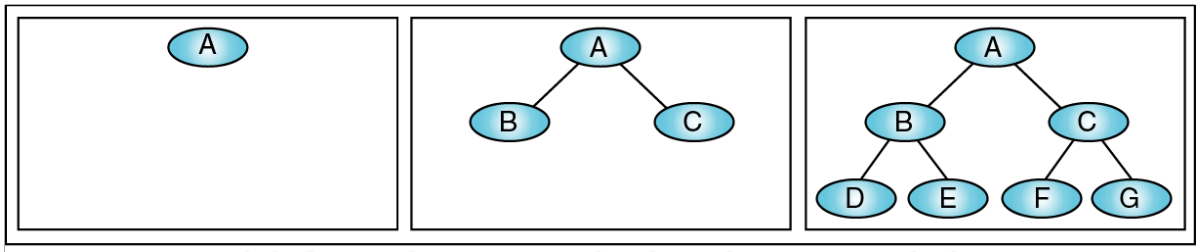


- Ancestor 9 : 4, 2, 20
- Siblings : 9&1, 4&7, 2&8
- Leaves : 9, 1, 7, 3
- Internal nodes : 4, 2, 8

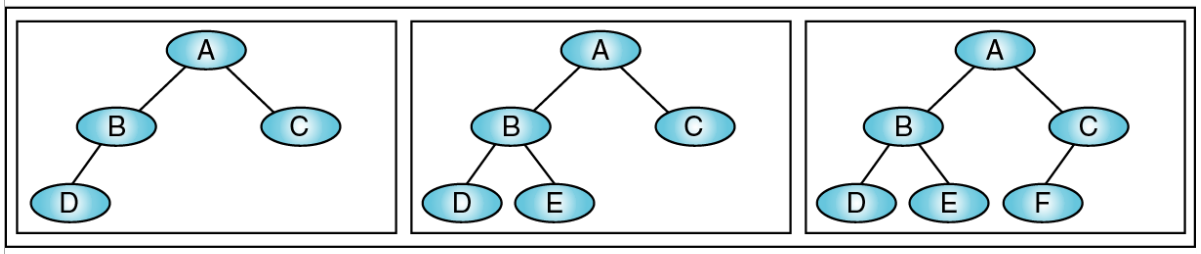


- In general, any nodes, N can be accessed by traversing the tree in the path, P start from root node. If the path, P consists of n, therefore node N is located in the $n^{th}$ level and the path, P length will be n.
- Example: Path to node 9
  - Starting from root, there are three paths 20->2, 2->4, 4->9.
  - Therefore, the node 9 is located in $3^{rd}$ level and the length of the path is 3.
- The height of binary tree = maximum level + 1.

- Complete Binary tree is a tree of which their leaves are at the same level and each node will have two children or none.

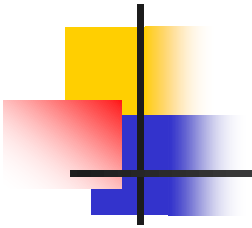



(a) Complete trees (at levels 0, 1, and 2)




(b) Nearly complete trees (at level 2)

**Fig. 5: Complete and nearly complete binary tree**



- A complete tree has the maximum number of entries for its height.
- To proof the complete binary tree:
  - ✓ Consider that the height of tree is k.

    It will contain;

    The number of Node: $2^k - 1$

    The number of Leaves: $2^{k-1}$
- Example:
  - ✓ Let say k = 4.

    Therefore, the number of nodes:

    $2^4 – 1=16 -1=15$ nodes.

    The number of leaves:

    $2^{4-1} = 2^3 = 8$ leaves.

- A Skewed Binary tree is a tree with only left or right children.

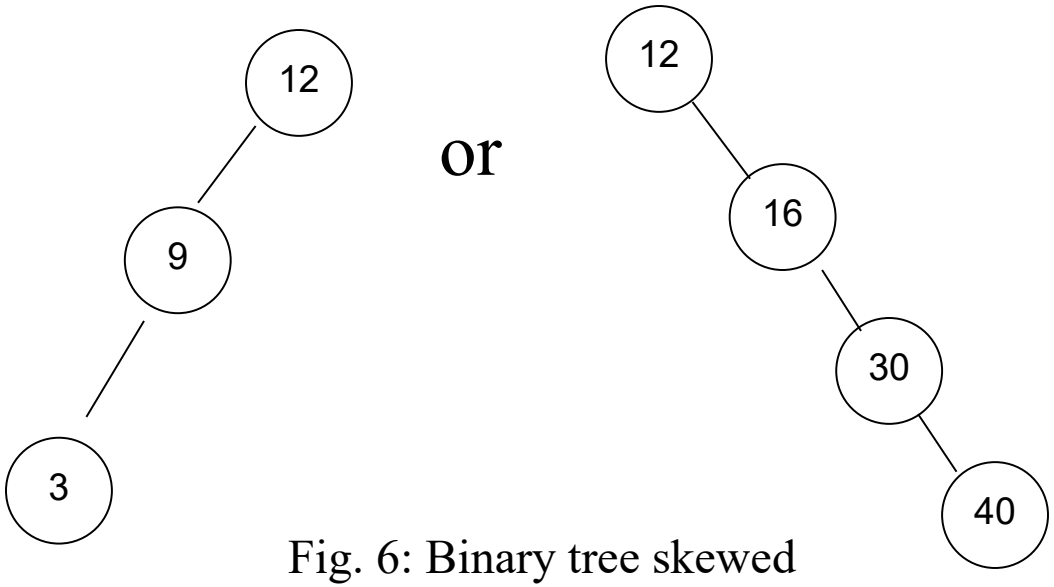
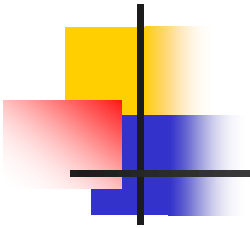
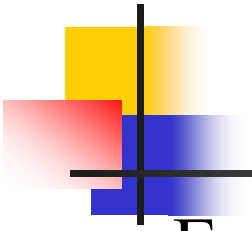



Fig. 6: Binary tree skewed



- **Note**:
  - ✓ In trees, there is only one path from root to each descendant. If there is more than one path, therefore the diagram is not a tree.
  - ✓ A diagram with their path in a circle and two different paths to a node but reconnected at another node is also not a tree.



# 6.3 Expression Tree

- Expressions tree is an application of binary tree.
- Arithmetic expression is represented by a tree.
- An expression tree is a binary tree with the following properties:

  1) Each leaf is an operand.

  2) The root an internal nodes are operators.

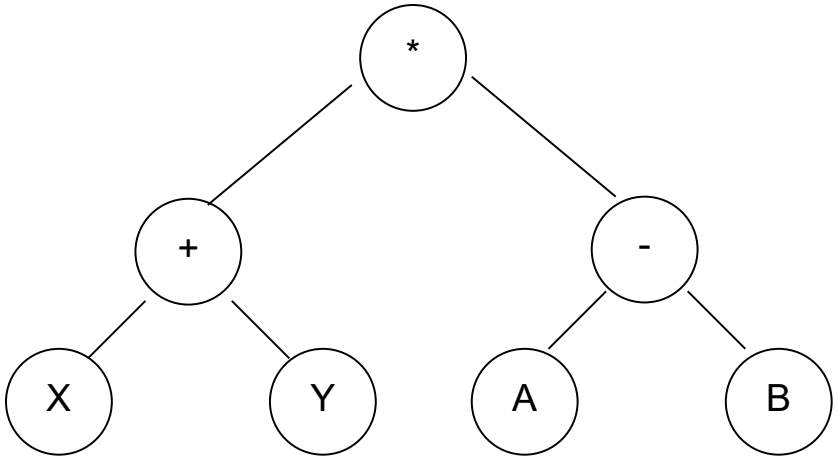
  (+, -, *, /)

  3) Subtrees are subexpressions with the root being an operator.

- Example:

  (X + Y) * (A - B)

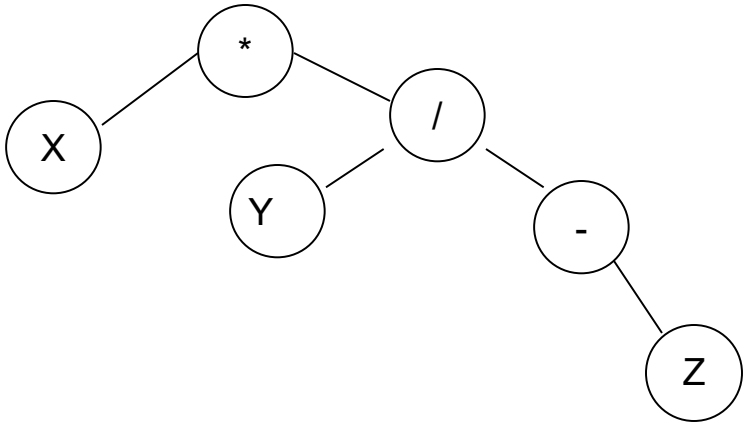
  Can be represented as:




- The parentheses (“(“ and “)”) is omitted but stated by the nature of the tree.

- Example:

  X * (Y / -Z)

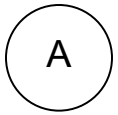


# 6.3.1 Build An Expression

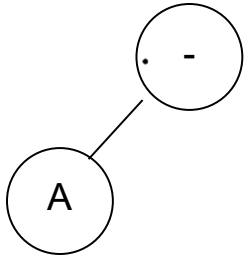
- Example :

  (A - B + C) * (-D)

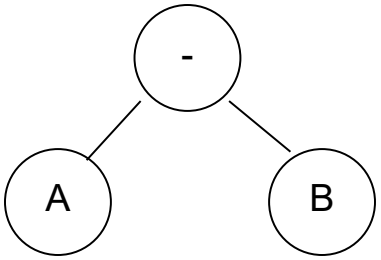
→ Build node A



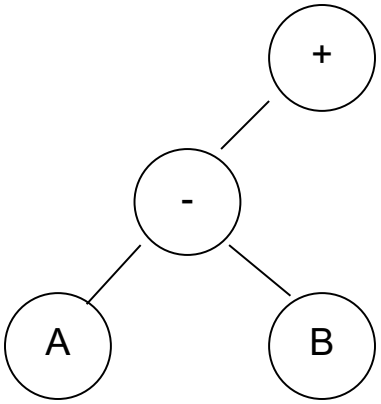

→ Build node -



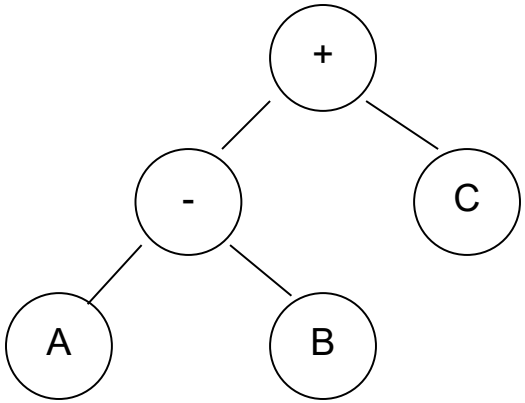
→ Build node B



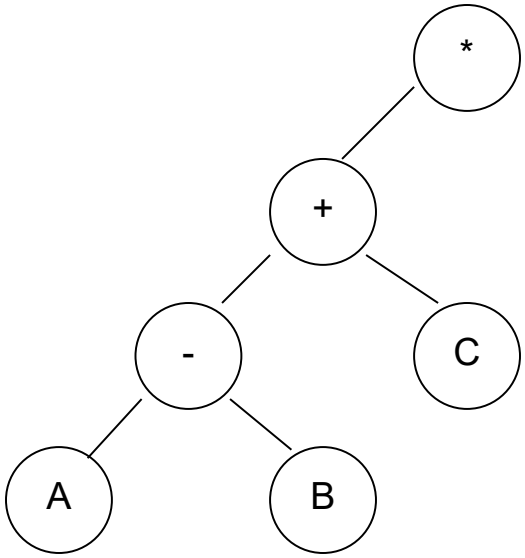

→ Build node +



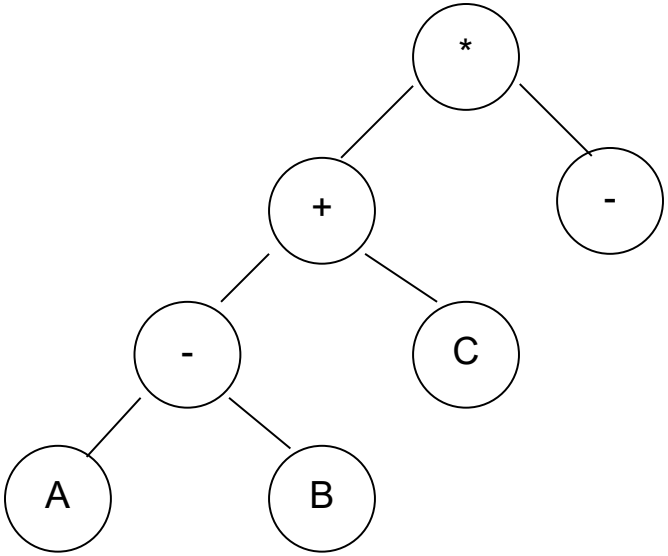
- Build node C



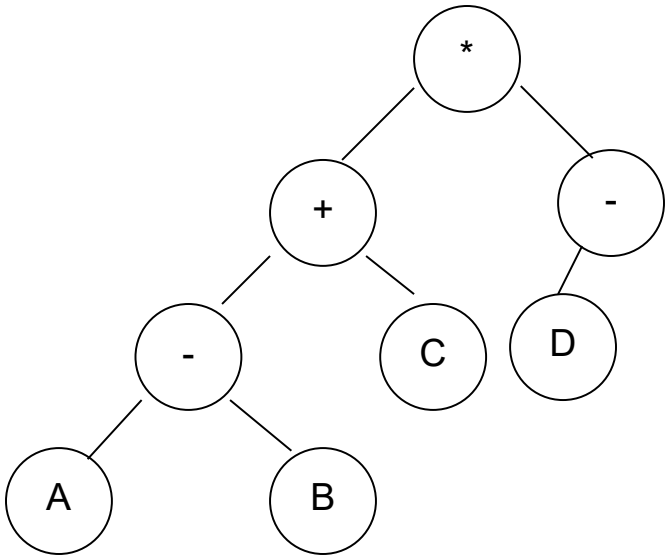

- Build node *



→ Build node –



→ Build node D



# 6.4 Binary Search Traversals

- A binary tree traversal requires that each node of tree be processed once and only once in predetermined sequence.
- There are 3 possible methods:

✓ Pre-order @ prefix → root - left - right

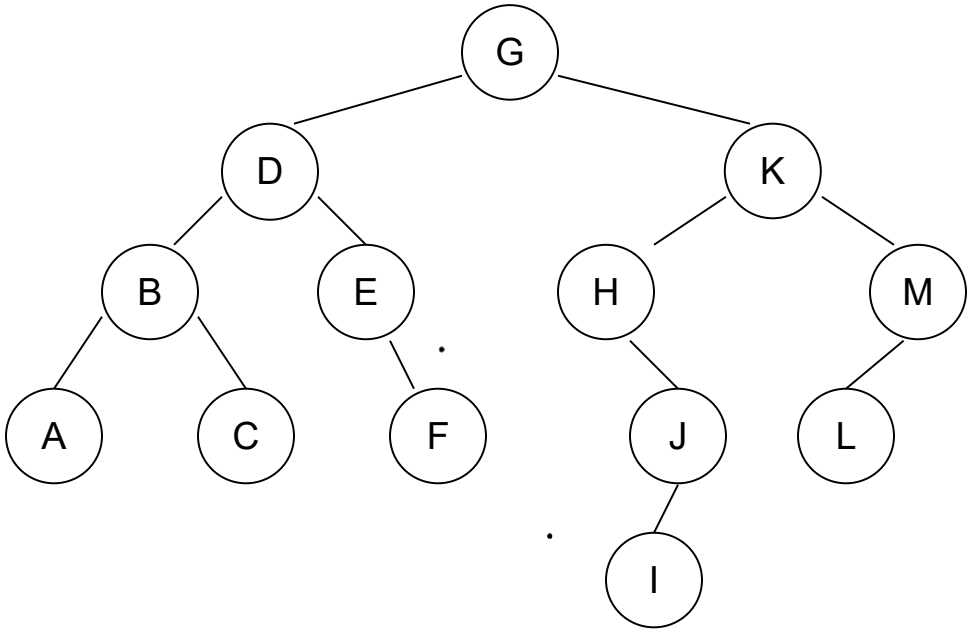
- ∏ visit root
- ∏ traverse left subtrees
- ∏ traverse right subtrees

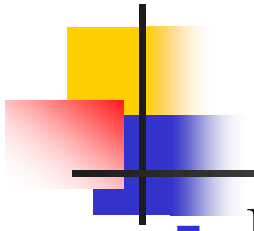
✓ In-order @ infix → left - root - right

- ∏ traverse left subtrees
- ∏ visit root
- ∏ traverse right subtrees

- ✓ Post-order @ postfix → left - right - root
  - ∏ traverse left subtrees
  - ∏ traverse right subtrees
  - ∏ visit root

- Example :





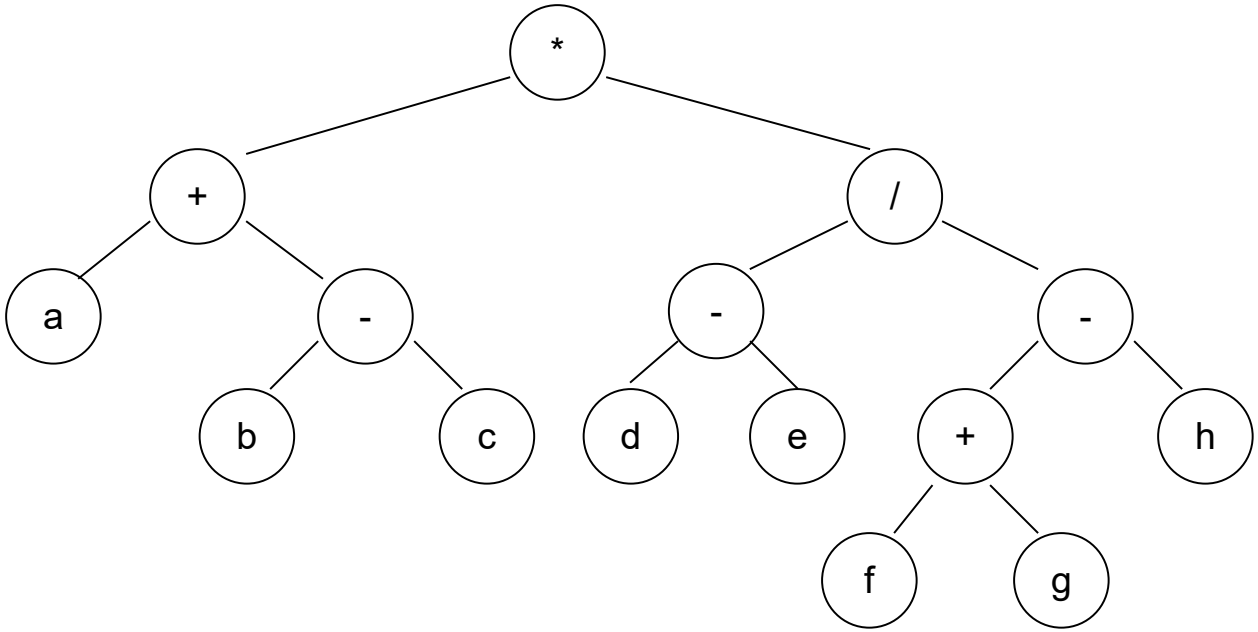
- Preorder (root - left - right)

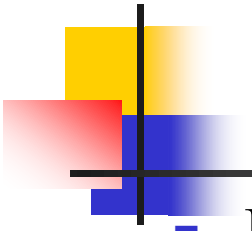
  G D B A C E F K H J I M L
- Inorder (left - root - right)

  A B C D E F G H I J K L M
- Postorder (left - right - root)

  A C B F E D I J H L M K G

- Example :



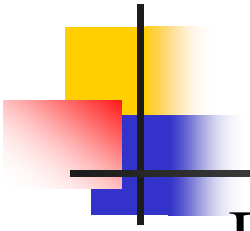


- Preorder (root - left - right)

  * + a - b c / - d e - + f g h
- Inorder (left - root - right)

  [a + (b – c)] * [(d – e) / (f + g – h)]
- Postorder (left - right - root)

  a b c - + d e - f g + h - / *



# 6.5 Binary Search Tree

- Binary Search Tree (BST) is a tree with the following properties:

  1. All items in the left subtree are less than the root.
  2. All items in the right subtree are greater than or equal to the root.
  3. Each subtree is itself a binary search tree.

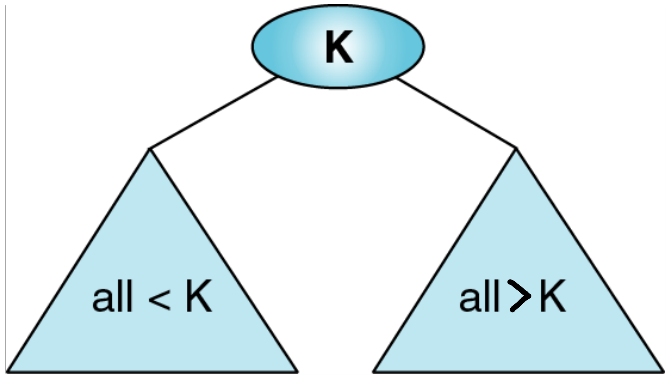


Fig 6: A binary search tree

- A BST is a binary tree in which the left subtree contains key values less than the root and the right subtree contains key values greater than or equal to the root.

- Binary Search Tree (BST) vs Binary tree (BT).

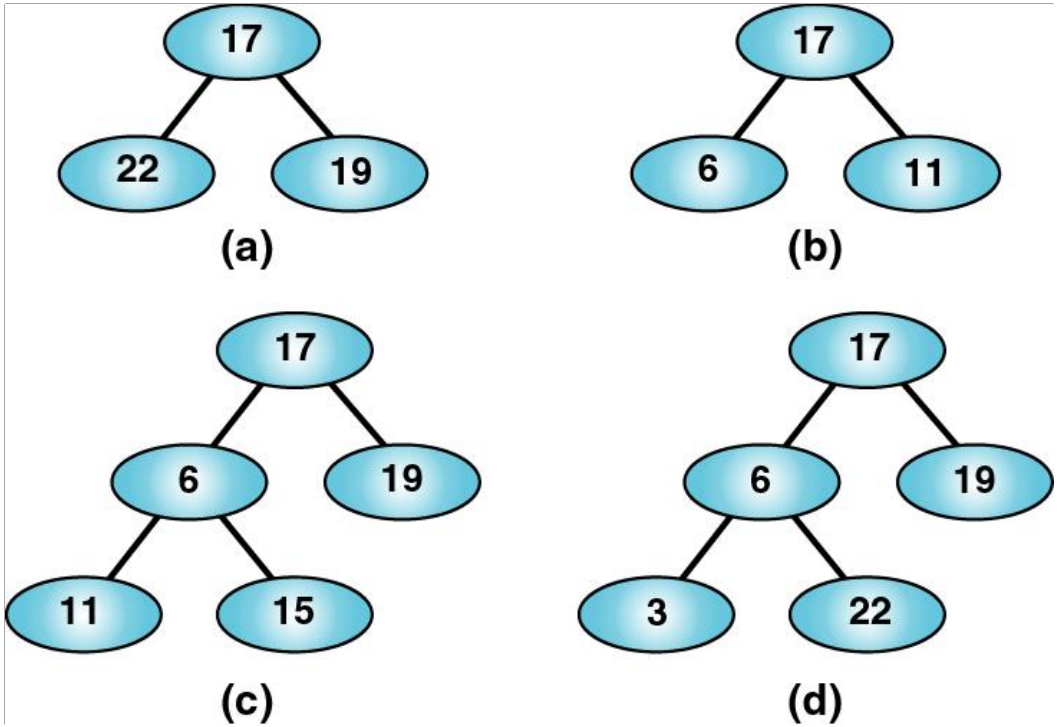
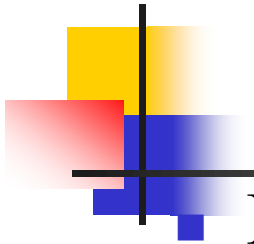



Fig. 7: Invalid binary search trees



- Basic operations:
  - ✓ Construction – build a null tree.
  - ✓ Destroy - delete all items in the tree.
  - ✓ Empty – check the tree is empty or not. Return TRUE if the tree is empty; return FALSE if the tree is not empty.
  - ✓ Insert – insert a new node into the tree.
  - ✓ Delete – delete a node from a tree.
  - ✓ Traversal – traverse, access, and process an item in the tree.
  - ✓ Search– search an item in the tree.

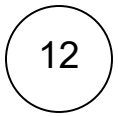
# 6.5.1 Binary Search Tree

- Example :

Process to create a tree;

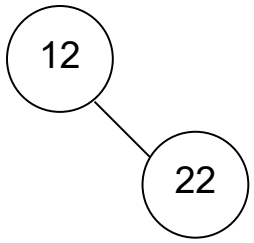
12 22 8 19 10 9 20 4 2 6

→ Insert node 12



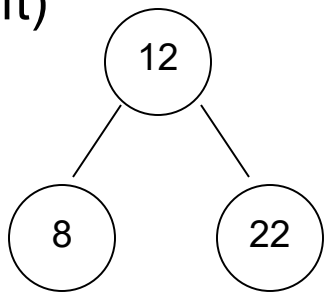

→ Insert node 22

(22 > 12 ⇒ Right)



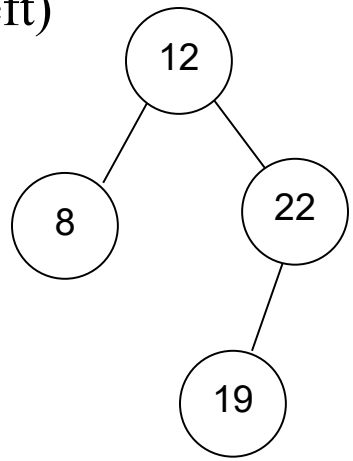
→ Insert node 8

(8 < 12 ⇒ left)



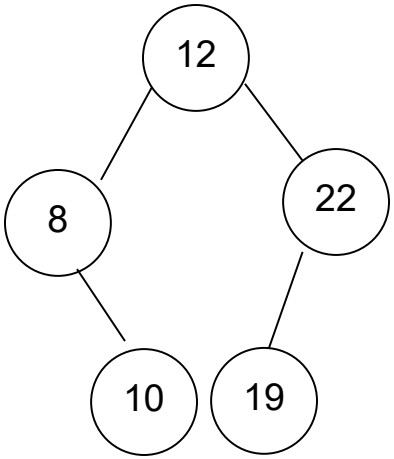

→ Insert node 19

(19 >12 ⇒ Right, 19 < 22 ⇒ left)



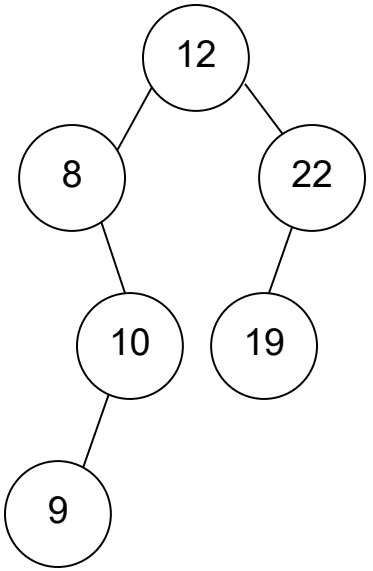
→ Insert node 10

(10 < 12 ⇒ left, 10 > 8 ⇒ Right)



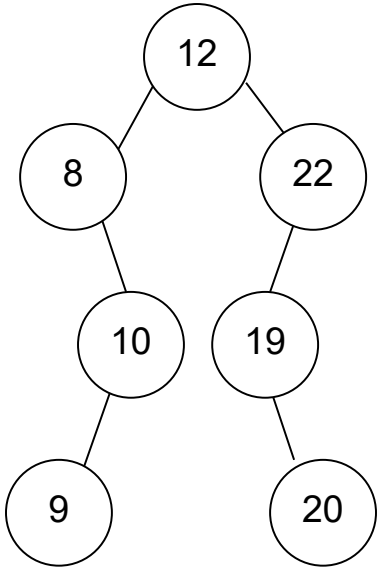

→ Insert node 9

(9 < 12 ⇒ left, 9 > 8 ⇒ Right, 9 < 10 ⇒ left)



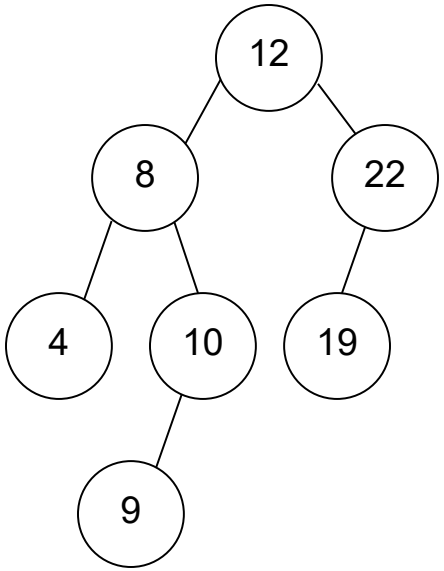
→ Insert node 20

(20 > 12 ⇒ Right, 20 < 22 ⇒ left, 20 >19 ⇒ Right)



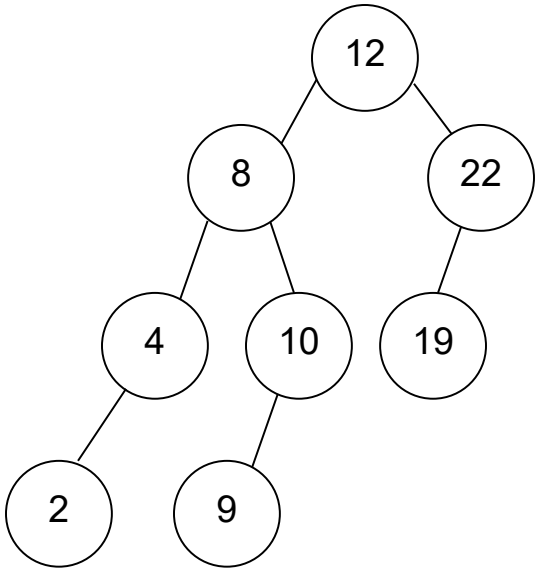
→ Insert node 4

(4 < 12 ⇒ left, 4 < 8 ⇒ left)



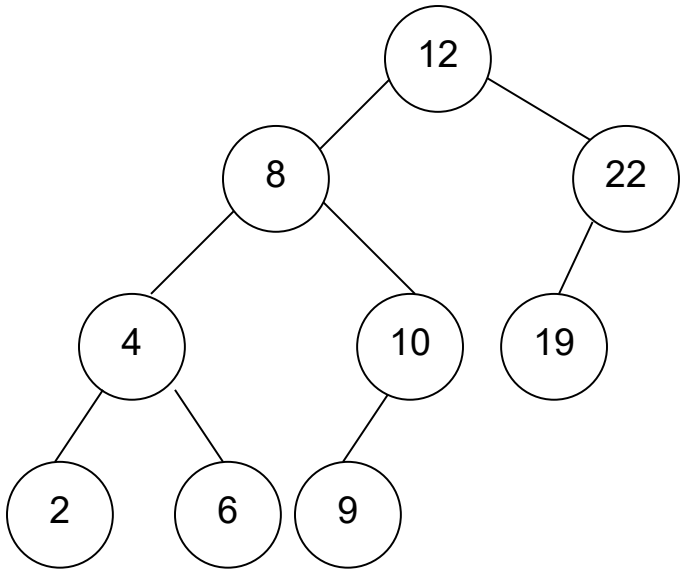
→ Insert node 2

(2 < 12 ⇒ left, 2 < 8 ⇒ left, 2 < 4 ⇒ left)



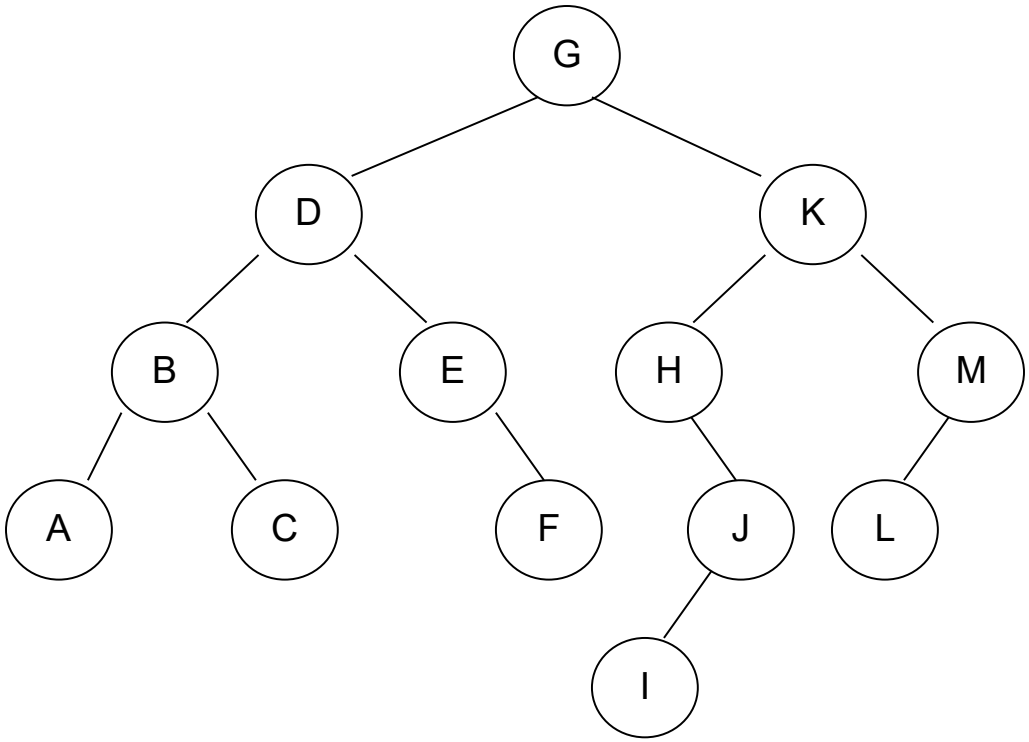
→ Insert node 6

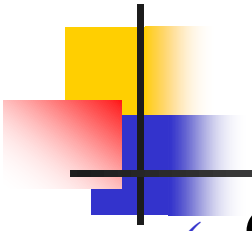
(6 < 12 ⇒ left, 6 < 8 ⇒ left, 6 > 4 ⇒ Right)



# 6.5.2 Delete A Node From Binary Search Tree

- To delete a node from a binary search tree, we must first locate it.
- There are four possible cases when we delete a node.





- Case 1:

  The node to be deleted has no children – leaves node

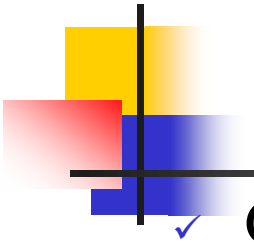
  (e.g. A, C, F, I, L).

  All we need to do is set the delete node's parent to null (e.g. B, E, J, M) and the leaves node will be deleted.

- Case 2:

  The node to be deleted has only a right subtree (e.g. E or H).

  If there is only a right subtree, then we can simply attach the right subtree to the delete node's parent.



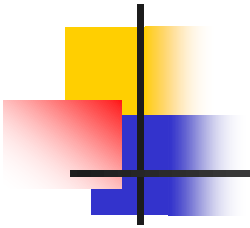
- Case 3 :

  The node to be deleted has only a left subtree (e.g. J and M).

  If there is only a left subtree, then we attach the left subtree to the delete node's parent.

- Case 4 :

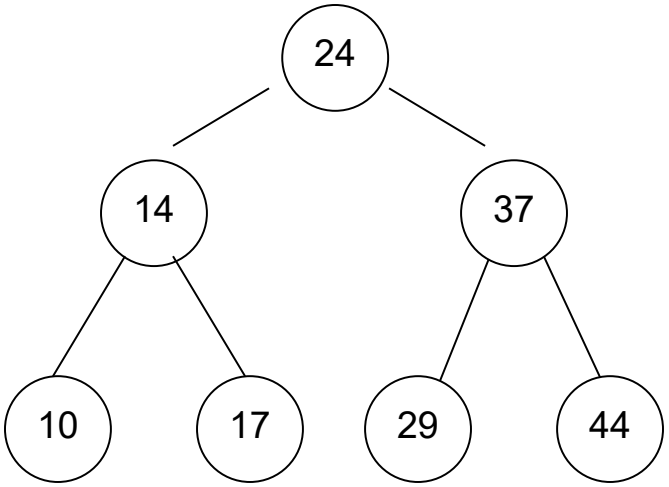
  The node to be deleted has two subtrees (e.g. B, D, G and K).

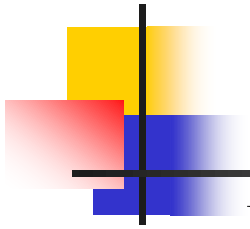


- We try to maintain the existing structure as much as possible by finding data to take the deleted data's place.
- This can be done in one or two ways:
  1. find the largest node in the deleted node's left subtree and move its data to replace the deleted node's data, or
  2. find the smallest node on the deleted node's right subtree and move its data to replace the deleted nodes data.

- Example :

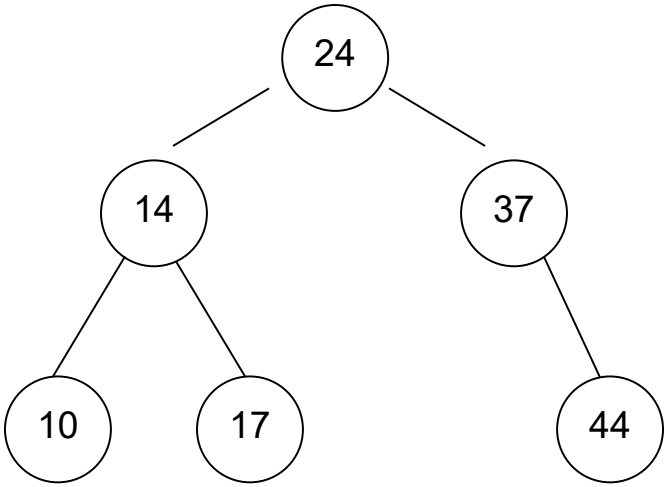
Show the process to delete 29, 37, and 24 from the following BST:





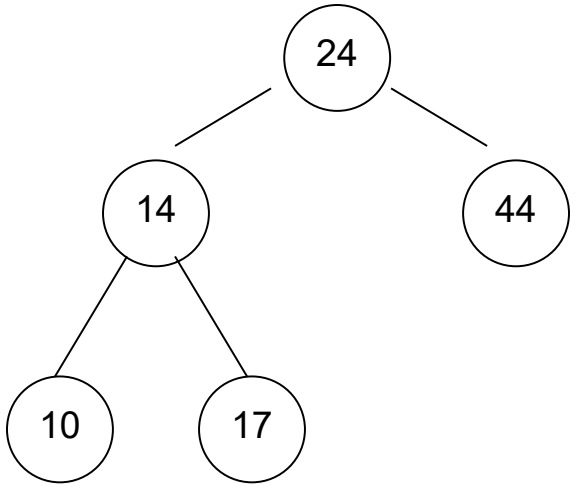
Delete node 29

29 $\Rightarrow$ leaf node, no need to change anything.



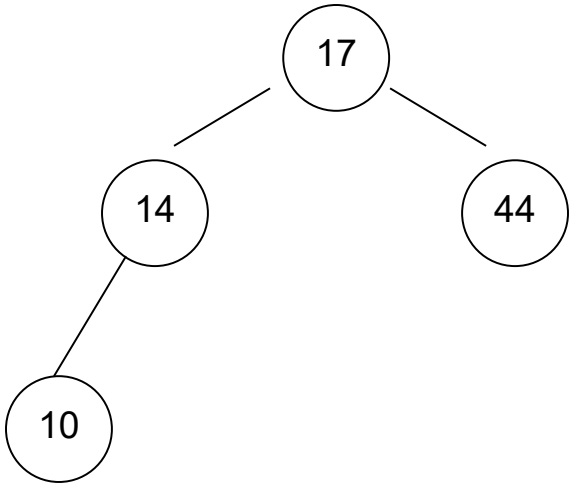
Delete node 37

37 $\Rightarrow$ there is no left node, change with the right node



Delete node 24

24 $\Rightarrow$ has two children, therefore search for inorder predecessor; 17 and replace with 24.



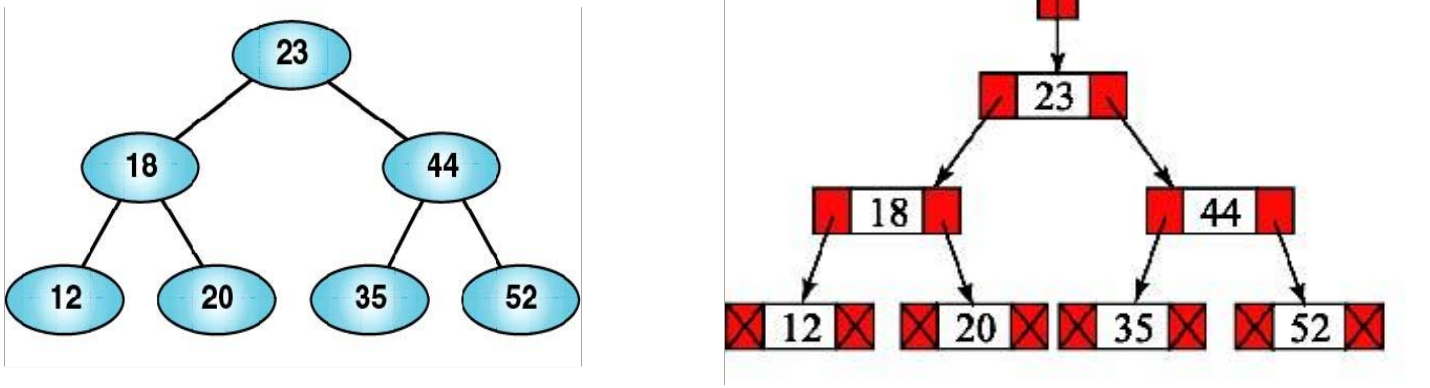
# 6.5.3 BST Class Implementation

```
class BSTNode
{
        Object root;
        BSTNode left, right;
        BSTNode (Object root) {
        this.root=root;
        }

        bool searchBST(BSTNode t, int n);
        void AddNode(BSTNode t, int newItem);
        bool isBSTEmpty (BSTNode t);
        void InsertNodeBST(BSTNode t, int newItem);
        void FindDescendant(BSTNode t, BSTNode q);
        void DeleteNodeBST(BSTNode t, int n);
        void destroyBST(BSTNode t);
        void preOrderBST(BSTNode t);
        void inOrderBST(BSTNode t);
        void postOrderBST(BSTNode t);
};
```

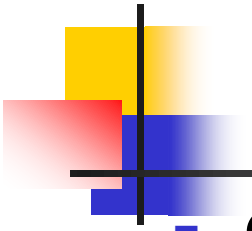
- The implementation file begins as follows:

  The implementations of the class's member functions are included at this point in the implementation file.




(a) Conceptual (b) Physical

Fig. 8: Conceptual and physical BST implementations



## Create BST

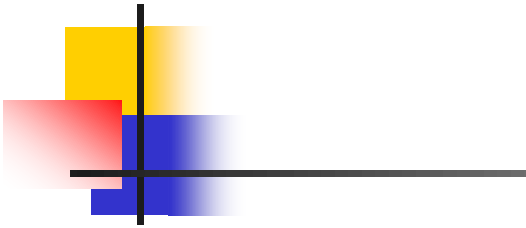
- Create BST, initialize the root (i.e. root) to NULL, this indicates as the new empty BST.
- Function definition:

```
public  BSTNode(Object root)
{
root = NULL;          // no memory allocated until node inserted
                        into BST
} // end constructor
```

## Destroy BST

- Destroy BST deletes all data in a BST and recycle their memory.
- Function definition:

```
void destroyTree (BSTNode pWalk)
{
      if (pWalk != NULL)
      {
           destroyTree(pWalk.leftP);
           destroyTree(pWalk.rightP);
           delete pWalk;pWalk = NULL;
       } // end if
} // end destroyTree
```



- **Empty BST**

- ✓ Empty BST is a method that returns a Boolean indicating if there is data in the BST or if it is empty. Thus, it returns TRUE if the BST is empty & FALSE if there is data.
- ✓ Function definition:

```
bool isEmpty ()
{
    return root == NULL;
 } // end isEmpty
```

## Insert

- Inserting a new node into a BST need to follow the left or right branch to down the tree until null subtree is found.

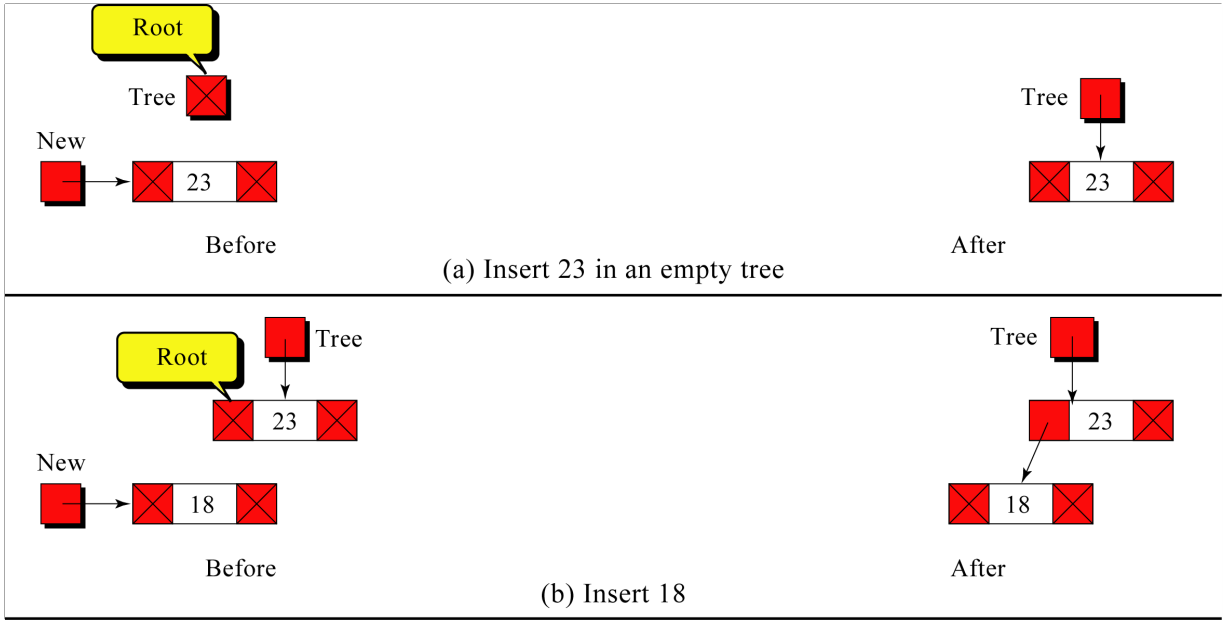
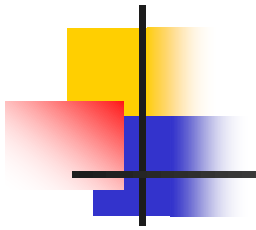



Fig. 9: Insert examples

Function definition:

```
 void insertBST (int newItem)
{
        bool found = false;
        pWalk = root;parent = NULL;
       for (;;)
       {
         if (found == true || pWalk ==NULL) break;
              parent = pWalk;
             if (newItem < pWalk.data)
                  pWalk = pWalk.leftP;
            else if (newItem > pWalk.data)
                 pWalk = pWalk.rightP;
            else
           found = true;
       } // end for
```



```
     if (found == true)
          System.out.println("Item already in the tree");
    else
    {
     pWalk = treeNode(newItem);
       if (parent == NULL)
          root = pWalk;
       else if (newItem < parent.data)
          parent.leftP = pWalk;
       else
          parent.rightP = pWalk;
     } // end else
} // end insertBST
```

```
// create new tree node
BSTNode treeNode(int newItem)
{
        pNew = new BSTNode();
        pNew.data = newItem;
        pNew.leftP = NULL;
        pNew.rightP = NULL;
        return pNew;
} // end treeNode
```

## Delete

- To delete a node from a BST, first the element must be found.
- Four possible cases need to be considered when deleting a node. (section 7.5.2)

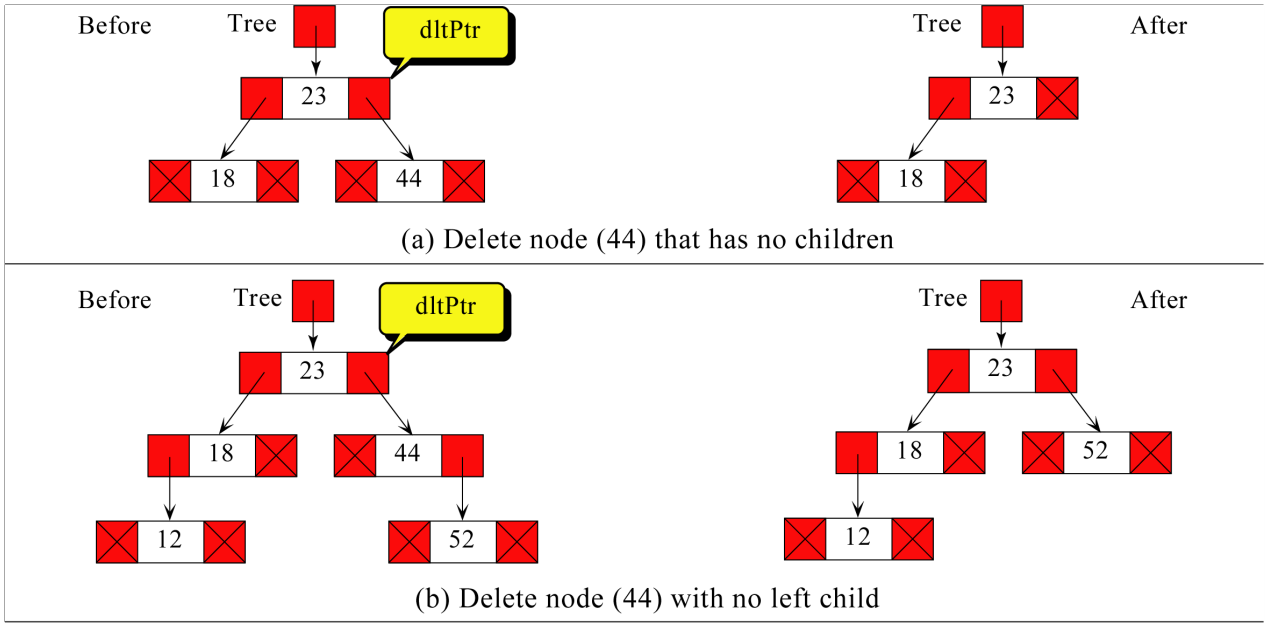
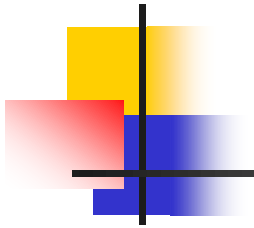


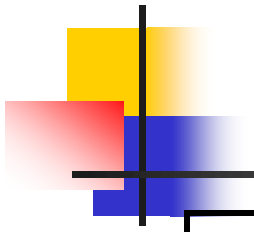

Fig. 10: Delete examples



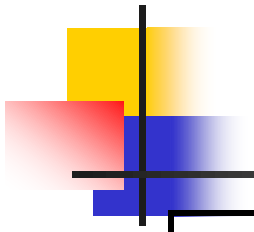
- Function definition:

```
 void deleteBST(int delItem)
{
      bool found;
      BSTNode x, parent;

     search2(delItem, found, x, parent);
     if (found == false)
    {
        System.out.println("Item not in the BST ");
        return;  // break;
    }
    // else – node has 2 children
```

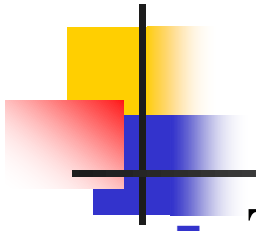


```
if (x.leftP != NULL && x.rightP != NULL)
{
        BSTNode xSucc = x.rightP;
        parent = x;
       while (xSucc.leftP != NULL)
       {
              parent = xSucc;
              xSucc = xSucc.leftP;
       }
      x.data = xSucc.data;
      x = xSucc;
} // end if
```



```
 // proceed with case where node has 0 or 1 child
 BSTNode subtree = x.leftP;
 if (subtree == NULL)
    subtree = x.rightP;
 if (parent == NULL)                  // root being deleted
    root = subtree;
else if (parent.leftP == x)          // left child or parent
     parent.leftP = subtree;
 else
     parent.rightP = subtree;   // right child or parent
 delete x;
} // end deleteBST
```

```
void search2 (int delItem, bool found, BSTNode locptr, BSTNode parent);
{
        locptr = root;
        parent = NULL;
        found = false;
        for (;;)
        {
                if (found == true || locptr == NULL) return;
               if (delItem < locptr.data)
               {
                    parent = locptr;
                    locptr = locptr.leftP;
               } // end if
              else if (delItem > locptr.data)
              {
                    parent = locptr;
                    locptr = locptr.rightP;
             } // end else if
             else
                  found = true
        } // end for
} // end search2
```

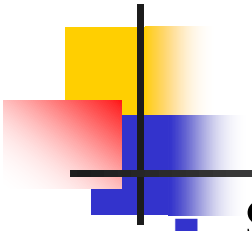


- **Traverse**

- ✓ Traversing algorithm is moving through the BST and visiting each node exactly once.
- ✓ There are three ways of doing a traversal namely preorder traversal, inorder traversal and postorder traversal (section 7.4).
- ✓ Function definition:

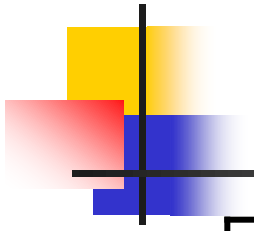
```
 void preorderTraverse ()   // preOrder traversal {
    preOrder(root);
} // end preorderTraverse
  void preOrder (BSTNode pWalk); {
      if (pWalk != NULL){
          System.out.print( pWalk.data + " ");
          preOrder (pWalk.leftP);
          preOrder (pWalk.rightP);
      }
} // end preOrder
  void inorderTraverse ( )  {  // inOrder traversal
      inOrder (root);
   } // end inorderTraverse
   void inOrder (BSTNode pWalk); {
      if (pWalk != NULL)
   {
```

```
        inOrder (pWalk.leftP);
        System.out.print( pWalk.data + " ");
        inOrder (pWalk.rightP);
        }
} // end inOrder
    void postOrderTraverse ( )   {    // postOrder traversal
         postOrder (root);
    } //end preorderTraverse
    void postOrder (BSTNode pWalk); {
        if (pWalk != NULL)
        {
            postOrder (pWalk.leftP);
            postOrder (pWalk.rightP);
            System.out.print( pWalk.data + " ");
        }
} // end preorder
```

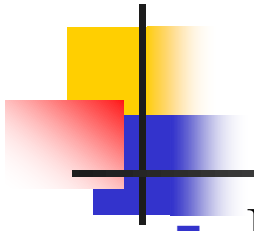


- **Search**

- ✓ Search algorithm is used to find a specific node in the tree.
- ✓ If the target value, newItem is greater than the root tree then the search is carry on the right subtrees, whereas if the target value, newItem is smaller than the root tree, search is carry on the left subtrees.
- ✓ Function definition:



```
bool searchBST (int targetItem);
{
        bool found = false;
        pWalk = root;
        for (;;)
        {
        if (found == true || pWalk == NULL) break;
        if (targetItem < pWalk.data)
            pWalk = pWalk.leftP;       // traverse to left subtree
        else if (targetItem > pWalk.data)
            pWalk = pWalk.rightP;      // traverse to right subtree
       else
           found = true;      // targetItem found
       } // end for
       return found;
} //end searchBST
```



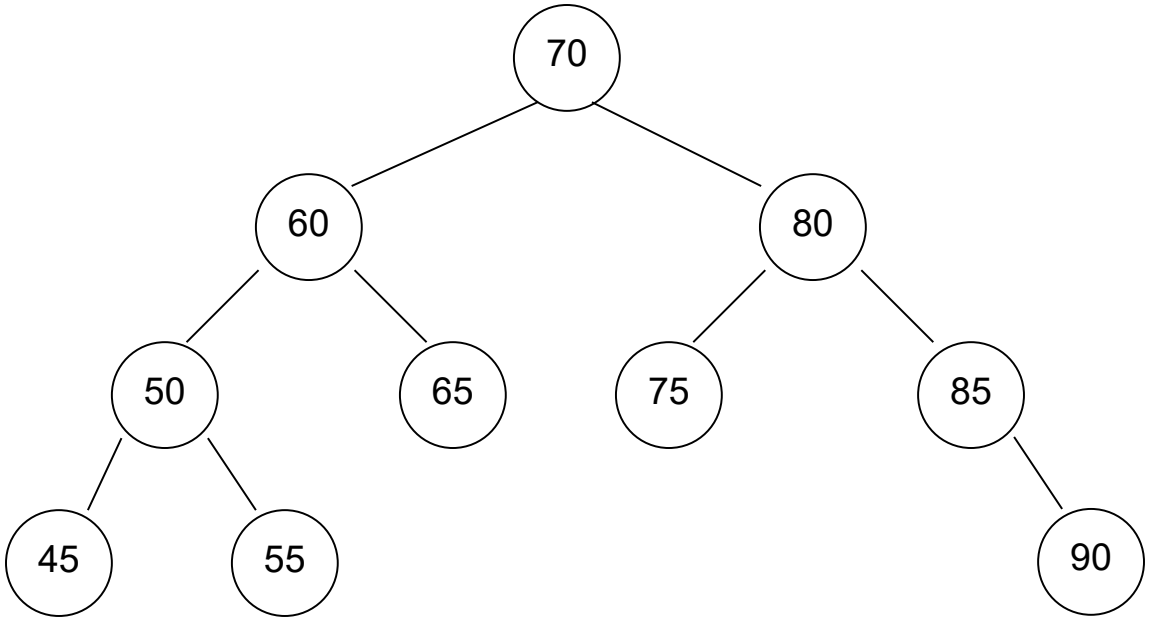
- **Exercises**

1.Create a binary search tree using the following data entered as a sequential set:

14, 23, 7, 10, 33, 56, 80, 66, 70

2. Insert 44 and 50 into the tree created in Q1.

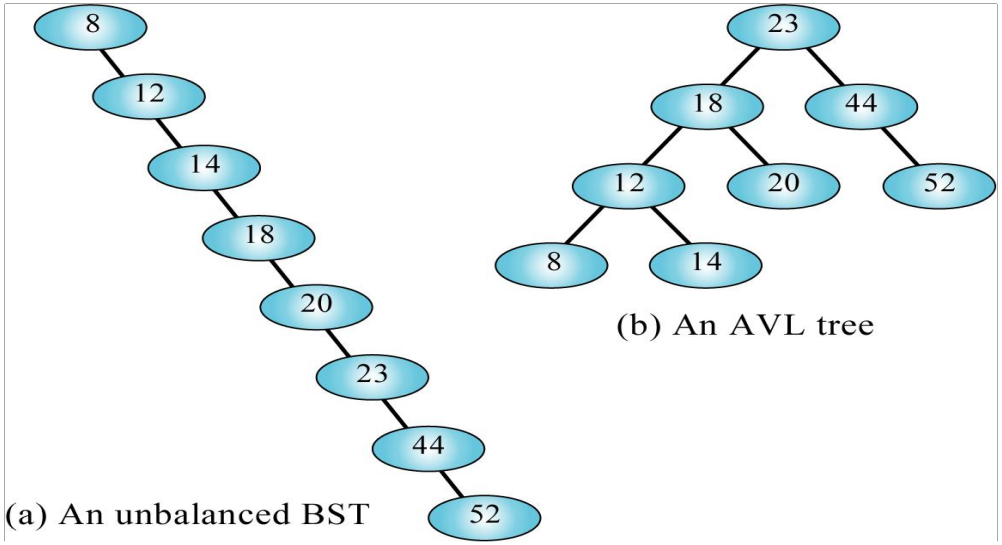
3. Delete the node containing 60 from the BST in the figure below.

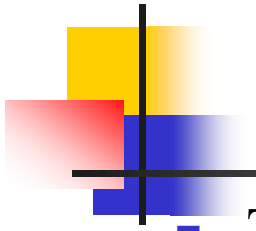



4. Delete the node containing 85 from the BST in above figure.

# 6.6 AVL Trees

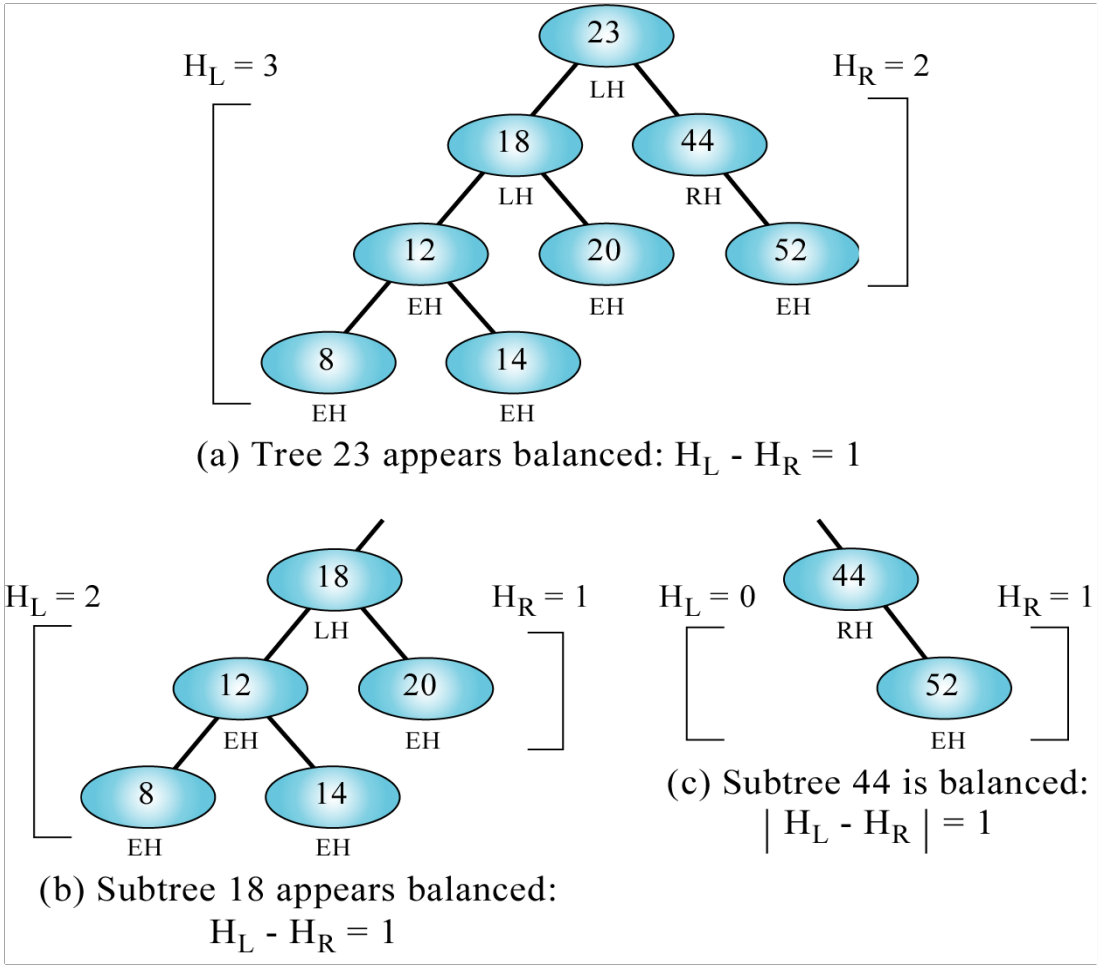
- AVL tree has been created by two Russian mathematicians; G.M. Adelson-Velskii and E.M.Landis in 1962.
- Known as *Height-Balanced Binary Search Tree*.
- AVL tree is a search tree in which the heights of the subtrees differ by no more than one.





- The heights of left subtrees and right subtrees are represented by HL and HR respectively.
- AVL Subtrees Height = | HL – HR | ≤ 1,

Therefore, AVL balance factor for each node is either 0, 1, @ -1.


$H_L = 3$
$H_R = 2$
23
LH
18
LH
44
RH
12
EH
20
EH
52
EH
8
EH
14
EH
(a) Tree 23 appears balanced: $H_L - H_R = 1$
$H_L = 2$
$H_R = 1$
18
LH
12
EH
20
EH
8
EH
14
EH
(b) Subtree 18 appears balanced:
$H_L - H_R = 1$
$H_L = 0$
$H_R = 1$
44
RH
52
EH
(c) Subtree 44 is balanced:
$| H_L - H_R | = 1$

# 6.6.1 Balancing Tree

- Rotation is a transformation process to convert unbalanced binary search tree to AVL tree.
- Unbalanced BST falls into one of this four cases:

✓ *Left of left*

- A subtree of a tree that is left high has also become left high.

✓ *Right of right*

- A subtree of a tree that is right high has also become right high.

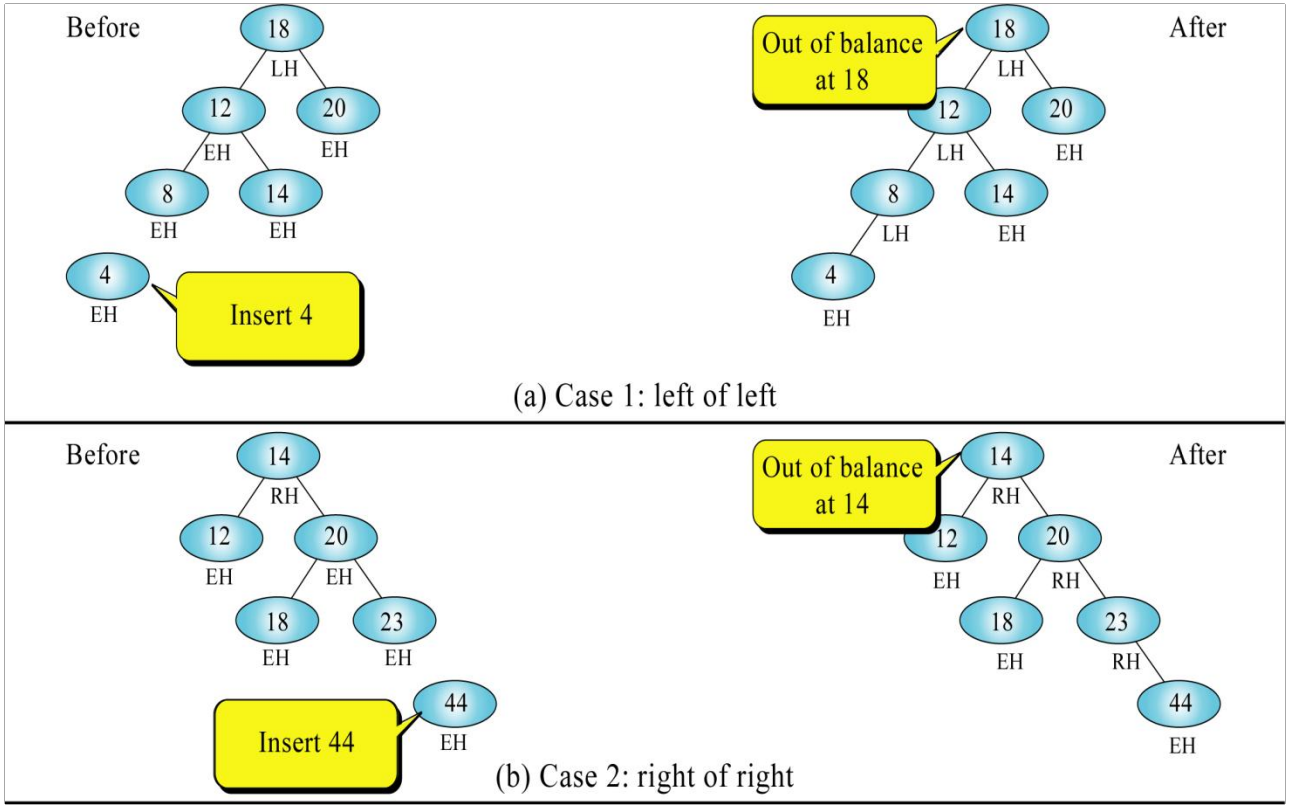
✓ *Right of left*

- A subtree of a tree that is left high has become right high.

✓ *Left of right*

- A subtree of a tree that is right high has become left high.

- These four cases are seen in figure 13 below:



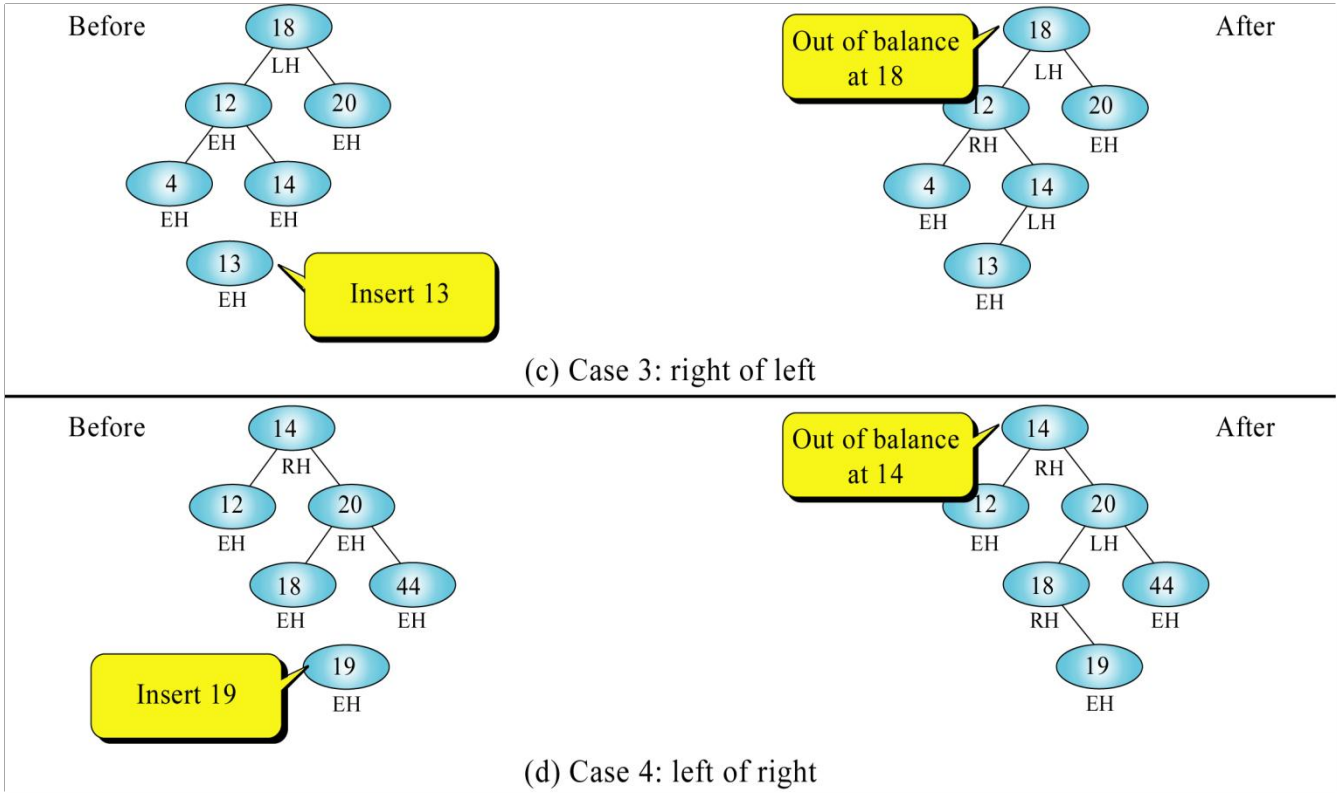
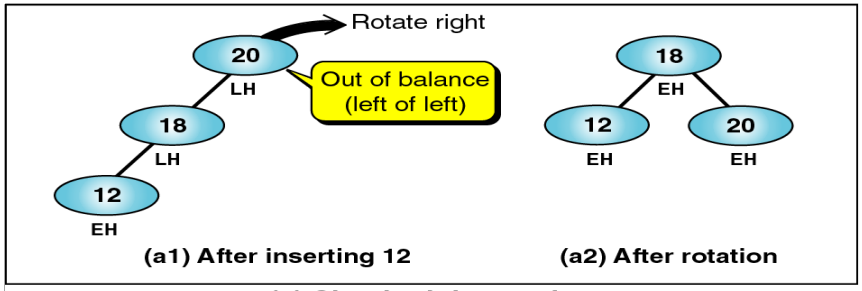
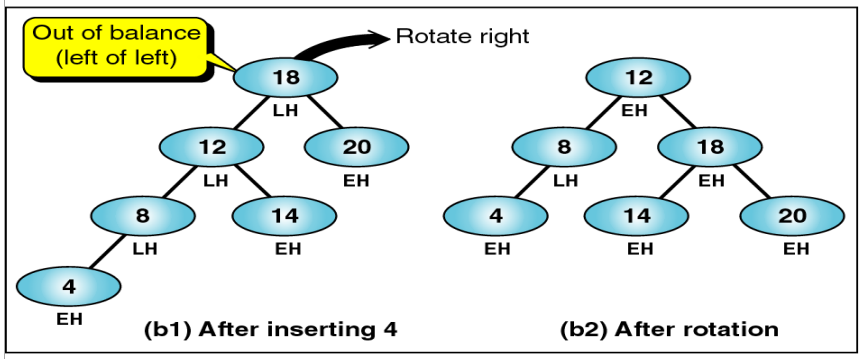



Fig. 13: Out of balance AVL trees

- Rotation transformation is implemented to overcome the four cases unbalanced trees:
  - ✓ *Left of left* –one way rotation to the right (*single right)* for out of balance node.




(a) Simple right rotation




(b) Complex right rotation

- *Right of right* – one way rotation to the left (*single left) for out of balance node.*

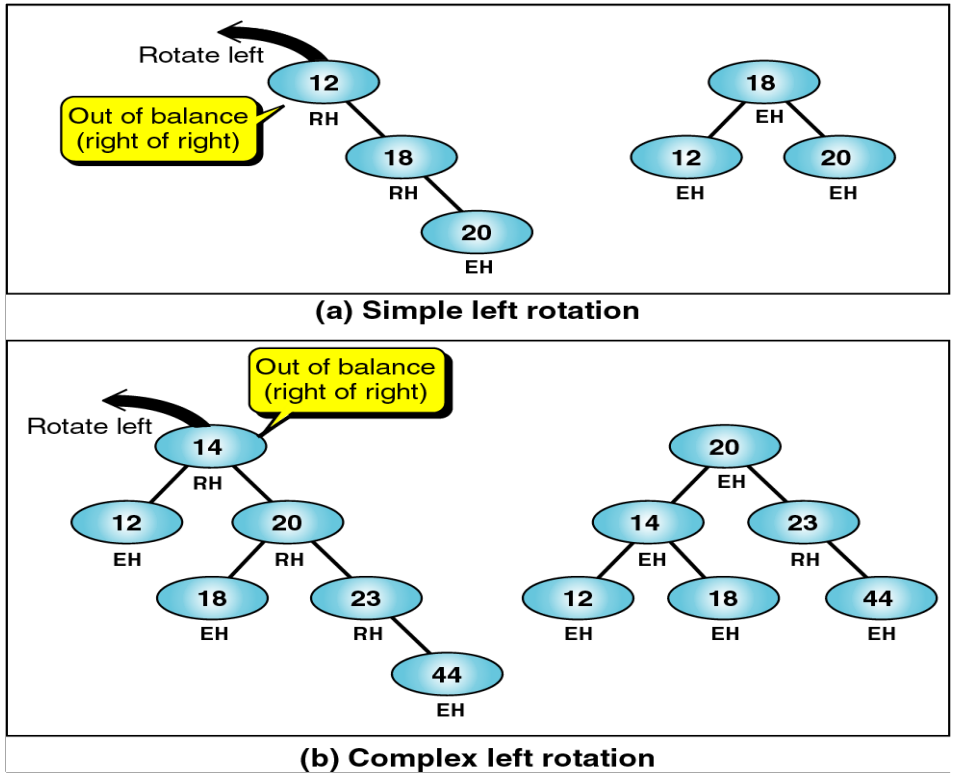
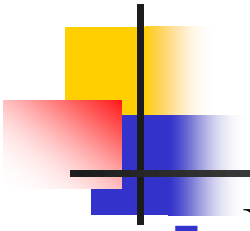



Fig 15: Right of right – single rotation left



- Note: The first two cases required single rotations to balance the trees. We now study two out of balance conditions in which we need to rotate two nodes, one to the left and one to the right, to balance the tree.
  - *Right of left* – one way rotation of left subtree to the left, followed by one way rotation of root (subtree) to the right (double right).

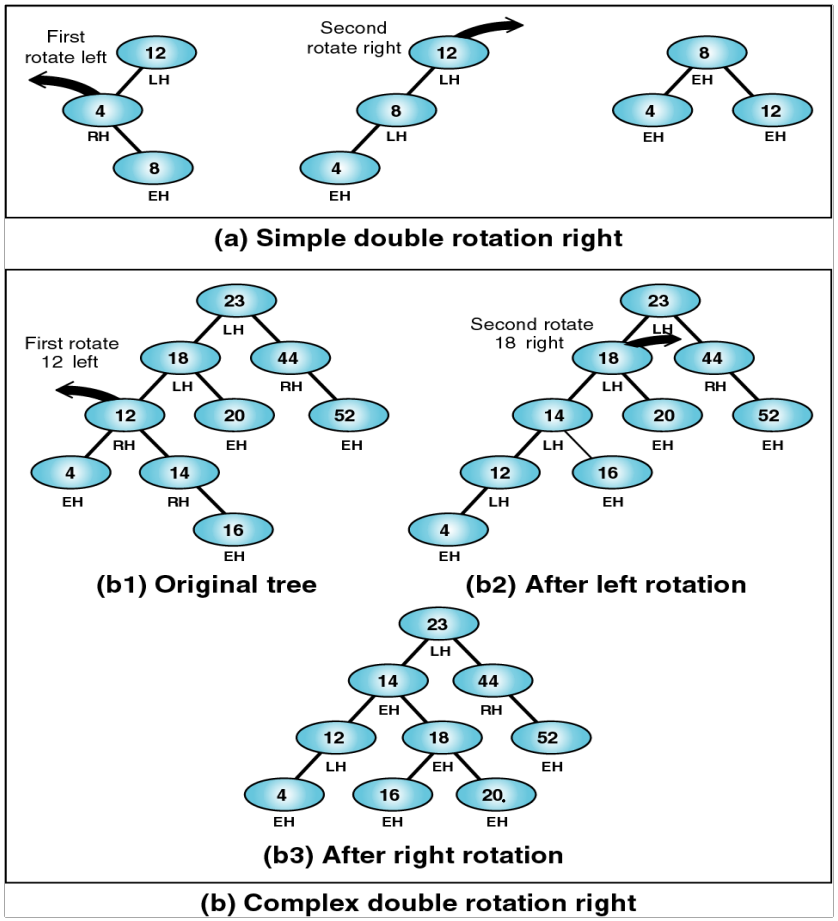
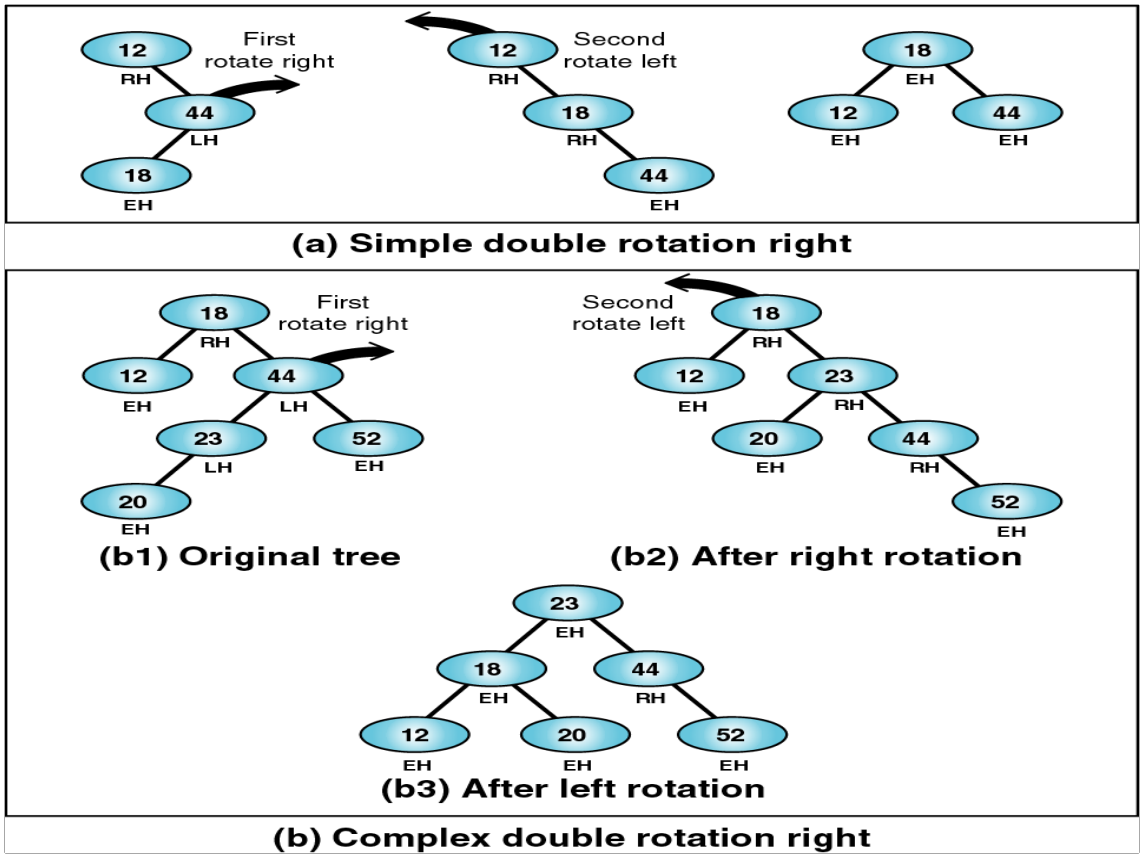



Fig. 16: Right of left – double rotation right

- *Left of right* – one way rotation of right subtree to the right, followed by one way rotation of root (subtree) to the left (double left).



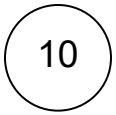
# 6.6.2 Building AVL Trees Using Insertion And Rotation

- Example:

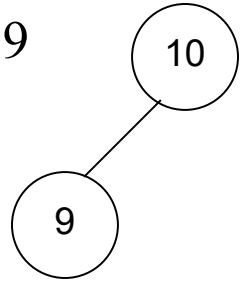
  Process to build an AVL tree

  10 9 5 12 3 7 25 40

→ Insert node 10

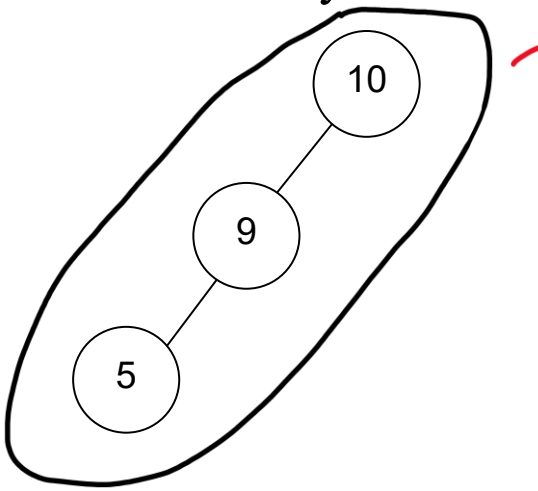


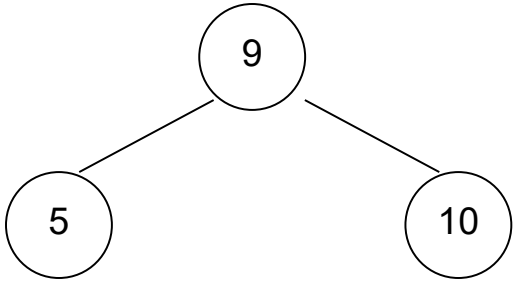

→ Insert node 9



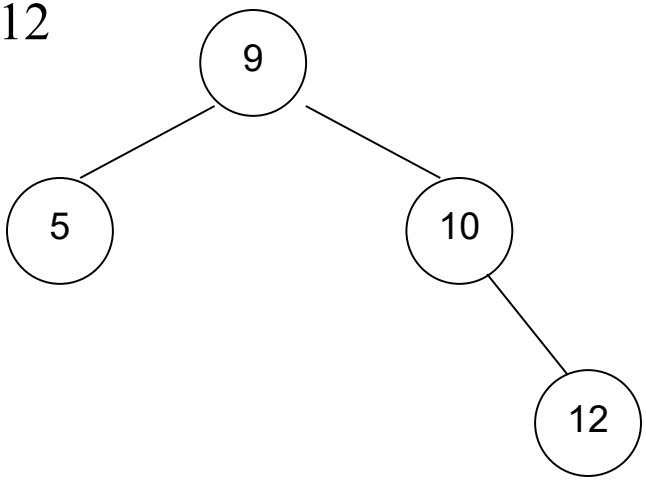
- Insert node 5

  (unbalanced tree ⇒ one way rotation to the right.)

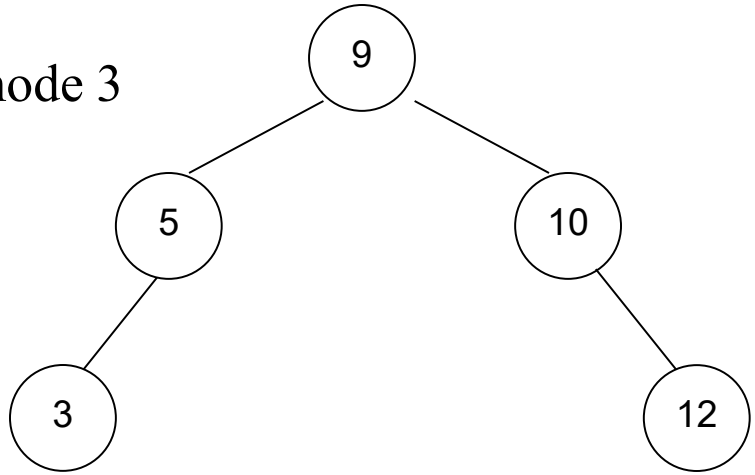


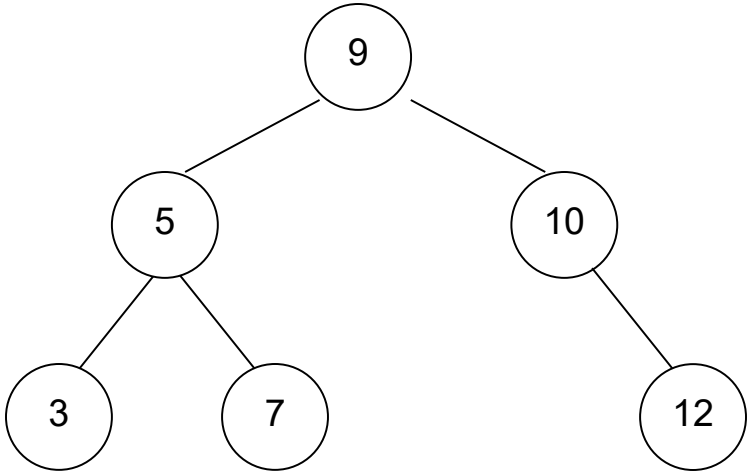
- Insert node 12



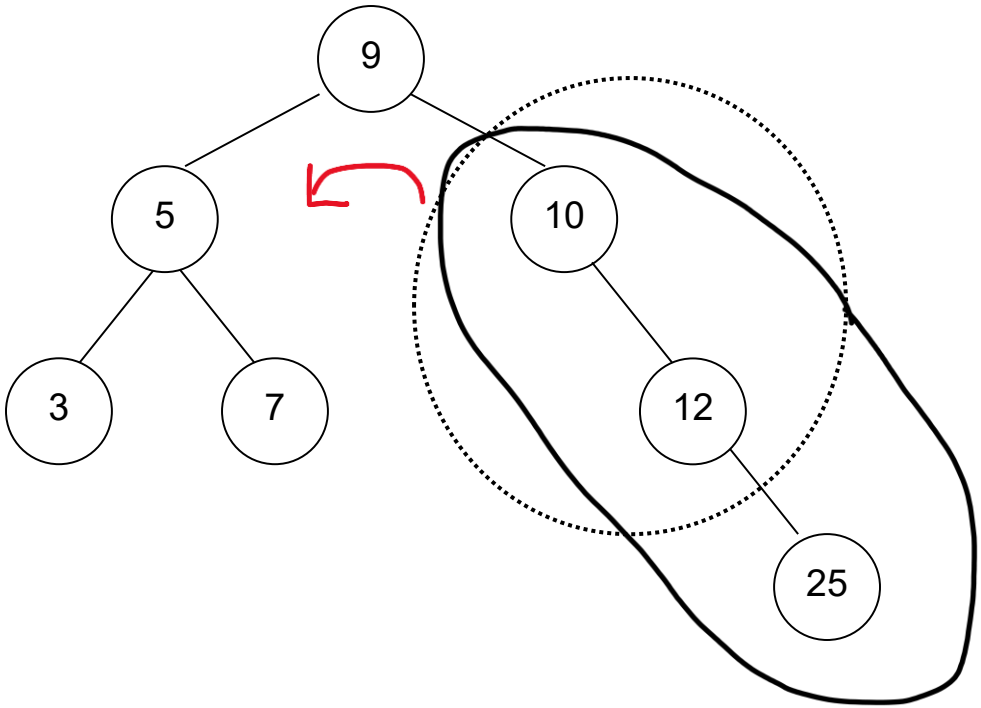

- Insert node 3

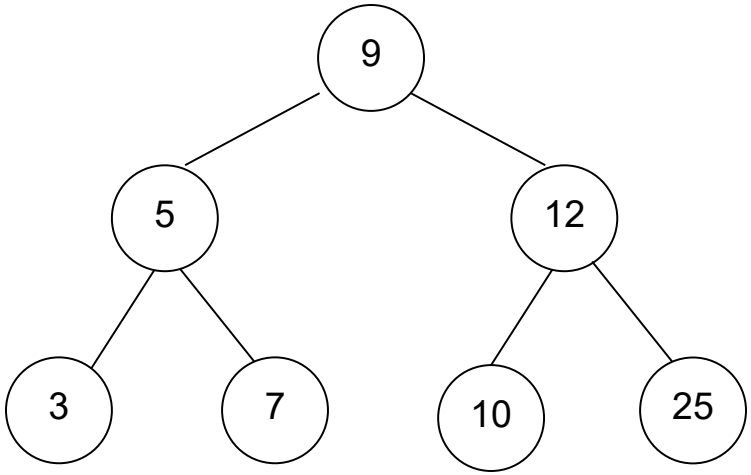


- Insert node 7

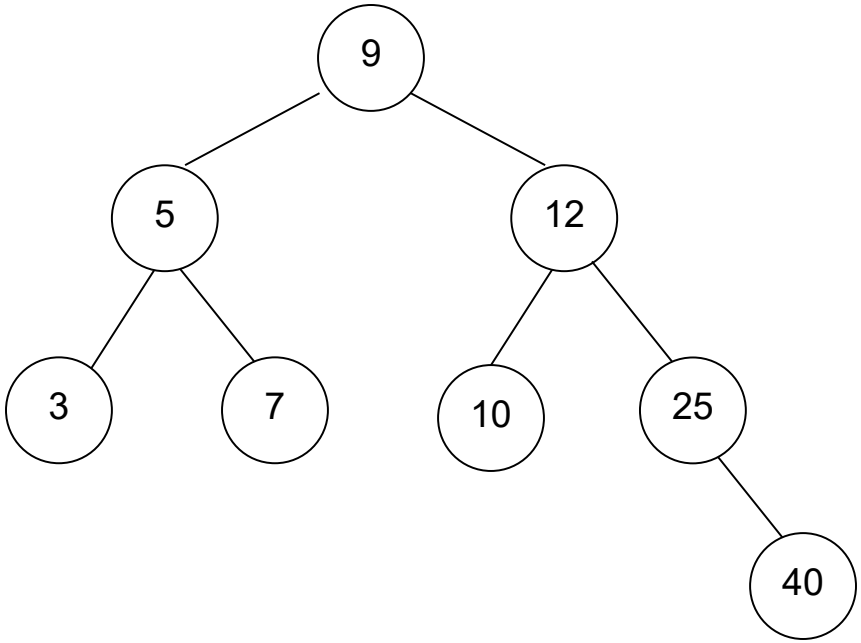


- Insert node 25
  (unbalanced tree ⇒ one way rotation to the left)



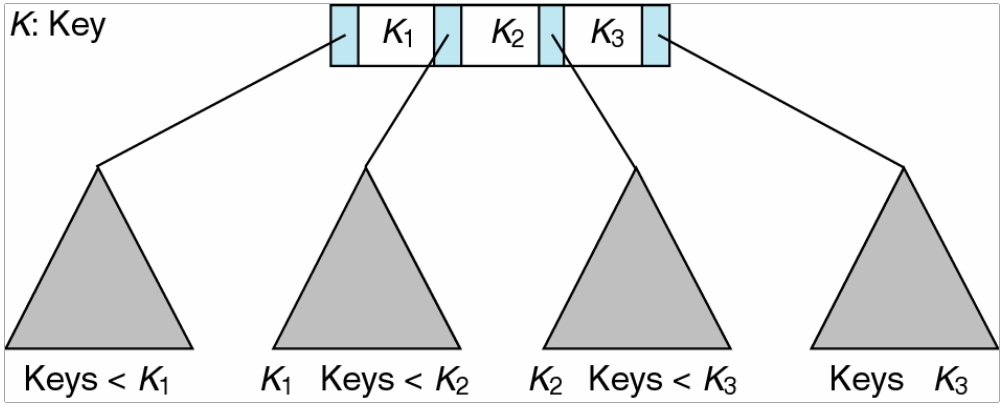

9
5
12
3
7
10
25

→ Insert node 40

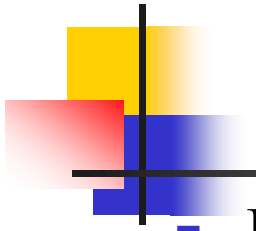


# 6.7 Multiways Tree : Pepohon B (B-Tree)

- An m-way tree is a search tree in which each node can have from 0 to m subtrees, where m is defined as the order of the tree.




- In 1970, two computer scientists working for Boeing Company have created a new tree structure called the B-tree.

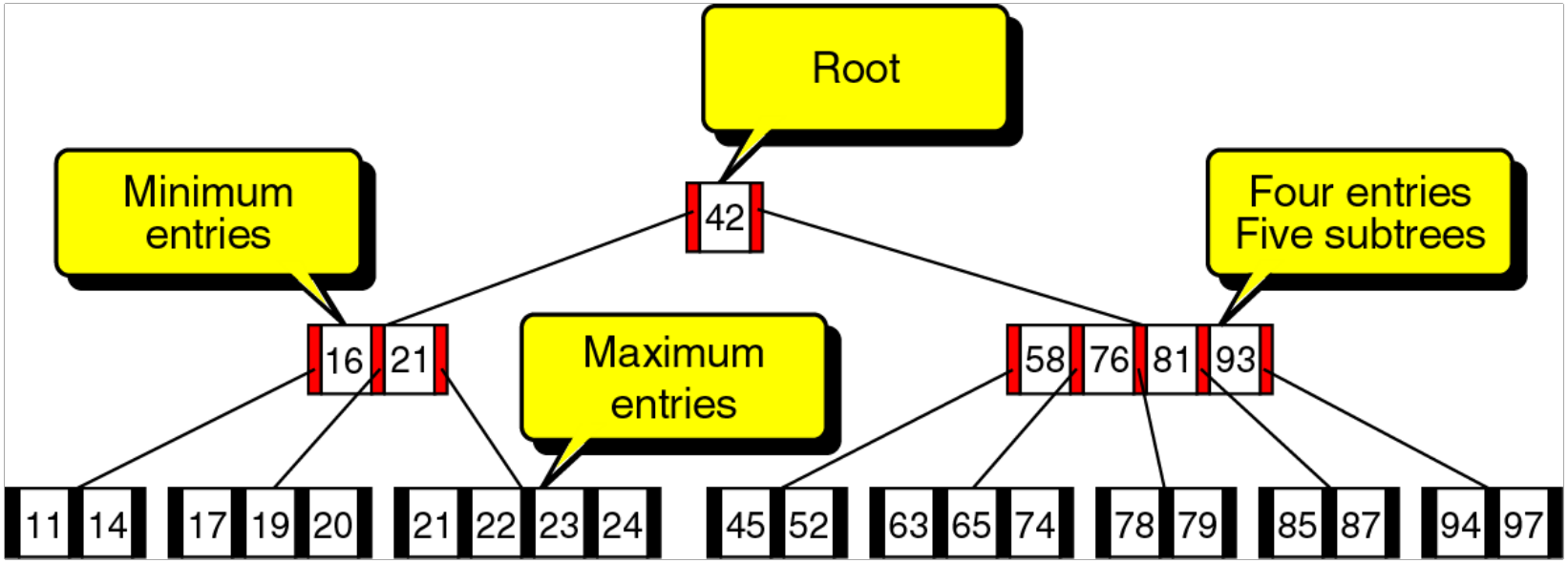


- B-Tree is an m-way search tree with the following properties:
- ✓ The root is either a leaf or it has 2 … m subtrees.
- ✓ All internal nodes have at least [m/2] non-null subtrees and at most m non-null subtrees.
- ✓ All leaf nodes are at the same level; that is, the tree is perfectly balanced.
- ✓ A leaf node has at least [m/2] – 1 and at most m – 1 entry.

- From the definition of a B-tree, it should be apparent that a B-tree is a perfectly balanced m-way tree in which each node with the possible exception of the root is at least half full.

Table 1: Entries in B-trees of various orders

| ORDER | Number of Subtrees | |
|---|---|---|
| | Minimum | Maximum |
| 3 | 2 | 3 |
| 4 | 2 | 4 |
| 5 | 3 | 5 |
| 6 | 3 | 6 |
| ..... | ..... | ..... |
| m | [m/2] | m |


Root
Minimum entries
42
Four entries Five subtrees
16 21
Maximum entries
58 76 81 93
11 14
17 19 20
21 22 23 24
45 52
63 65 74
78 79
85 87
94 97

# 6.7.1 Building A B-Tree (Of Order 5)

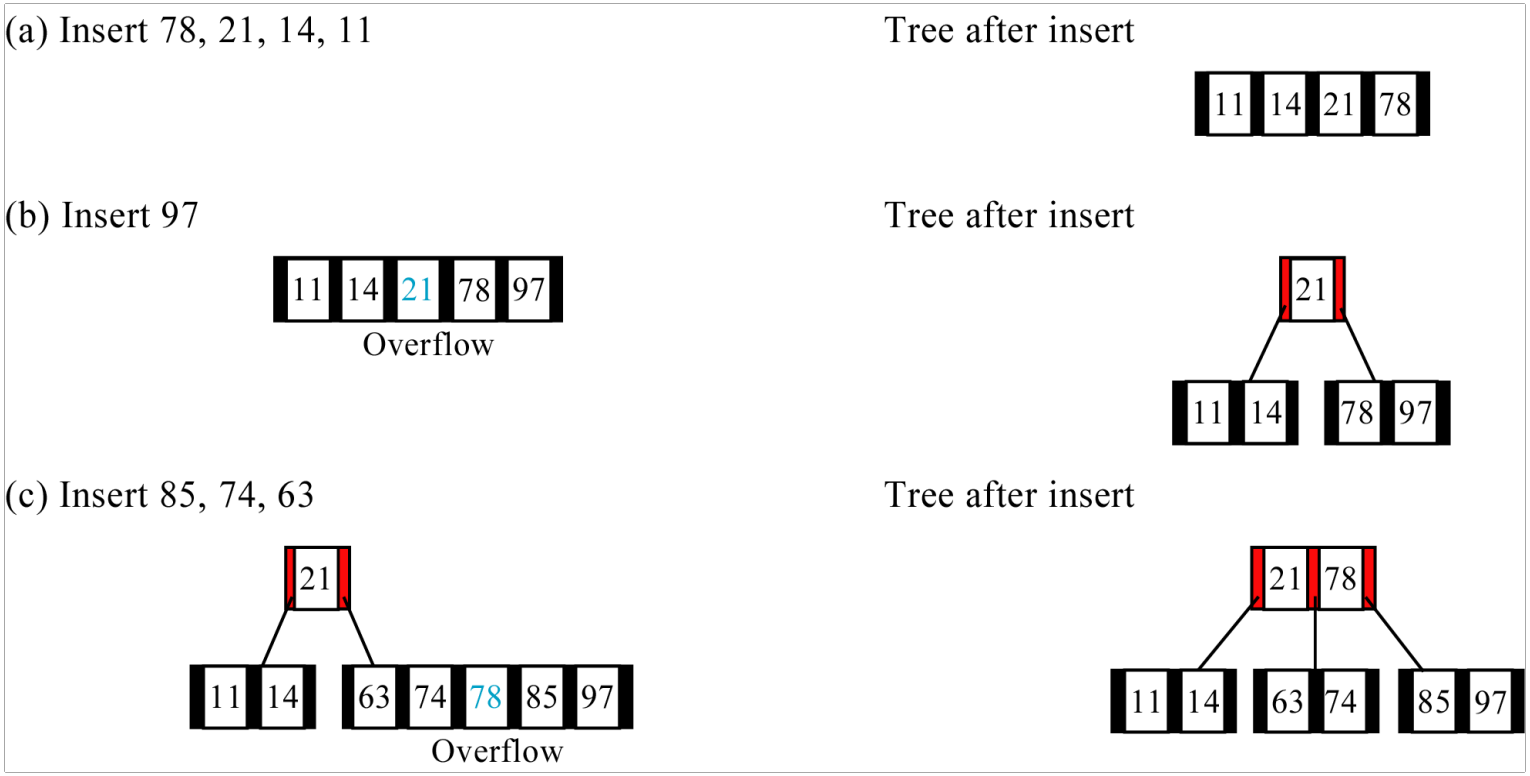



Fig. 19: Building a B-Tree of order 5

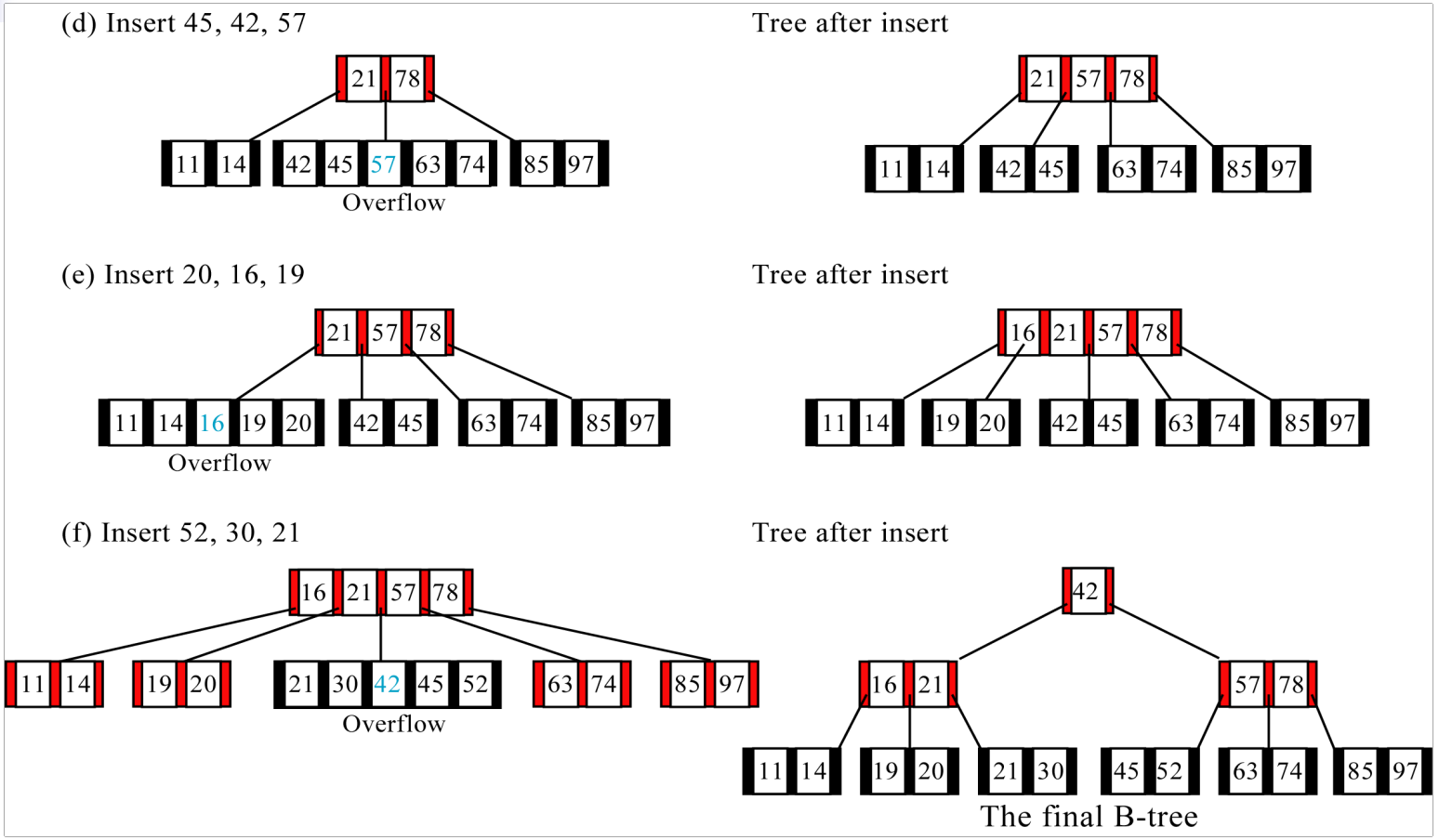



Fig. 20: Building a B-Tree of order 5

# 6.7.2 Deleting A Node In B-Tree

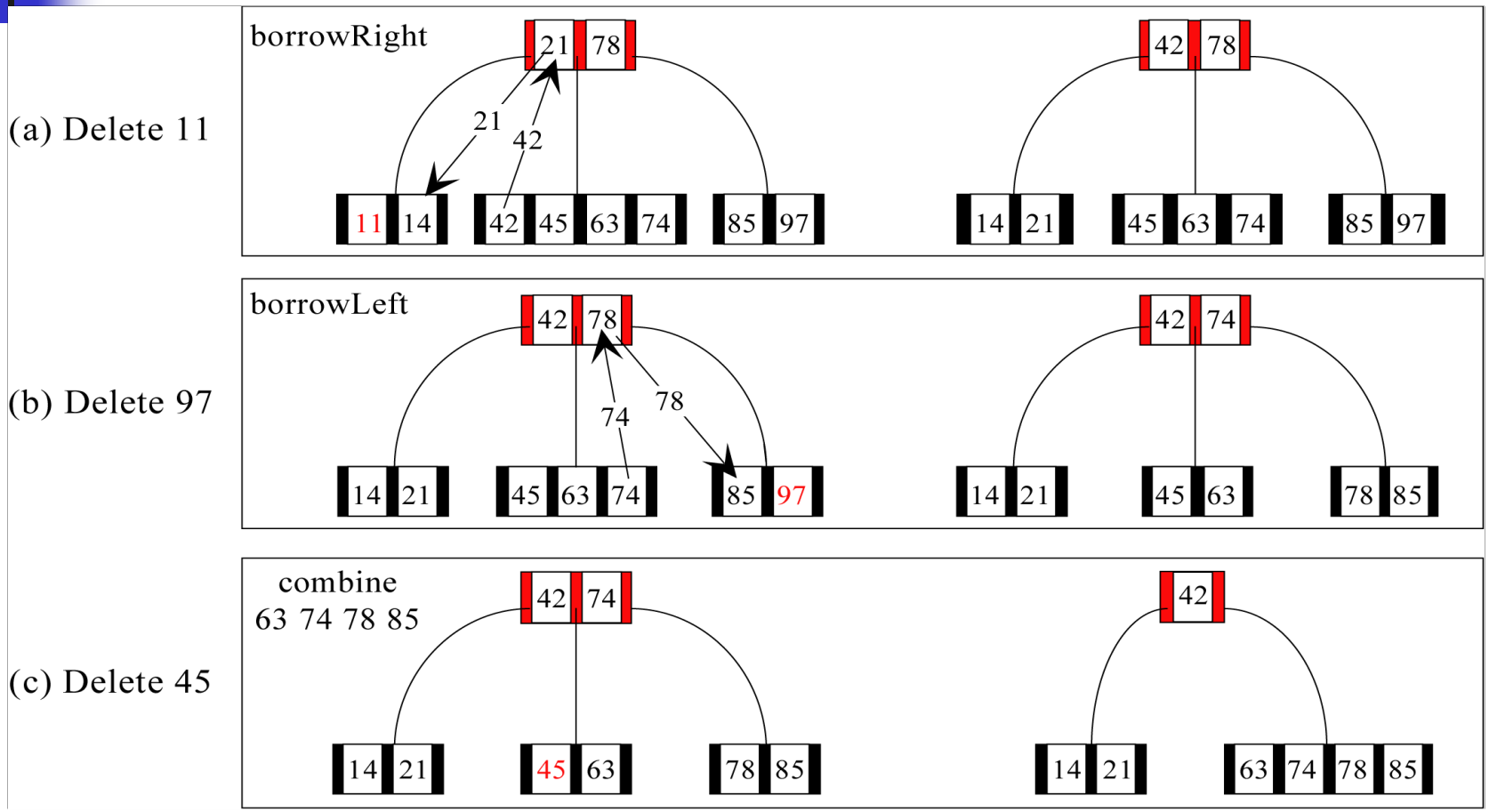



Fig. 21: B-tree of order 5 deletion summary

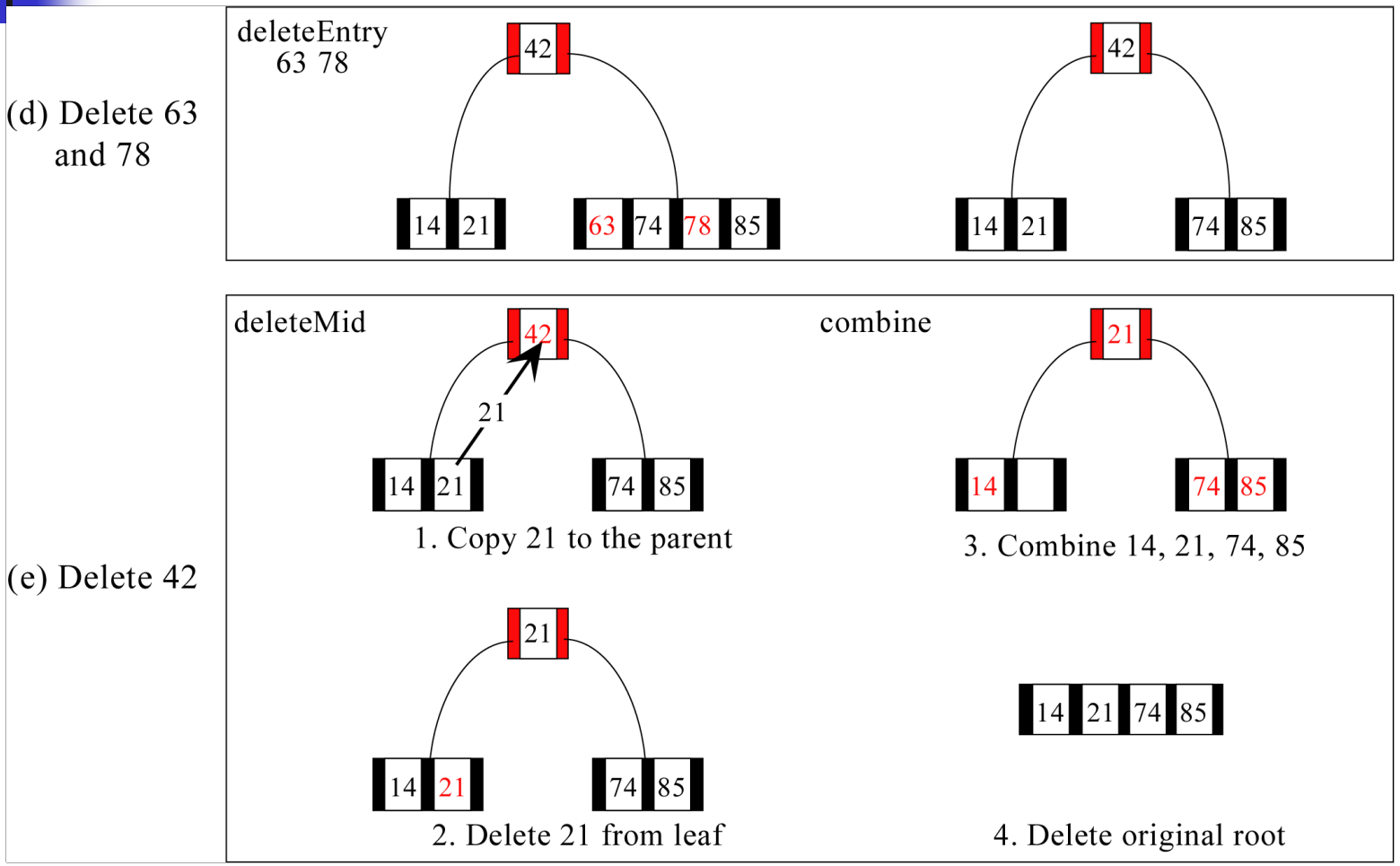
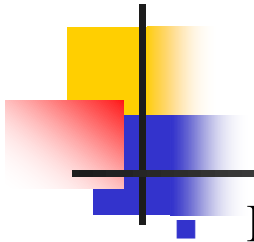



Fig. 22: B-tree of order 5 deletion summary



## Exercises

1. Draw the B-tree of order 3 created by inserting the following data arriving in sequence:

   92 24 6 7 11 8 22 4 5 16 19 20 78

2. Draw the B-tree of order 4 created by inserting the following data arriving in sequence:

   92 24 6 7 11 8 22 4 5 16 19 20 78

3. Using the B-tree of order 3 shown in below figure, delete 63, 90, 41, and 60, in each step, show the resulting B-tree.